SPŠT Střední průmyslová škola Třebíč

Maturitní práce

WEBOVÁ APLIKACE PRO PŘIHLÁŠKY DO DOMOVA MLÁDEŽE

Profilová část maturitní zkoušky

Studijní obor: Informační technologie
Třída: ITB4
Školní rok: 2025 Jan Prokůpek

Zadání práce

Střední průmyslová škola Třebíč
Manželů Curieových 734, 674 01 Třebíč

Zadání ročníkové práce

Obor studia: **18-20-M/01 Informační technologie**

Jméno a příjmení:	**Jan Prokůpek**		
Třída:	**ITB4**	Školní rok:	**2025/2026**
Číslo tématu:	**78**		
Název tématu:	**Webová aplikace pro přihlášky do domova mládeže**		
Rozsah práce:	**15 - 25 stránek textu**		

Specifické úkoly, které tato práce řeší:

Cílem práce je návrh a implementace webové aplikace pro evidenci přihlášek zájemců o ubytování v domově mládeže a pro podporu komunikace mezi žadateli a administrátory. Aplikace bude sloužit k centralizované správě procesu podávání žádostí a poskytne uživatelům přehledný a transparentní způsob sledování stavu jejich přihlášky.

Uživatelé budou mít možnost vytvářet a spravovat své žádosti prostřednictvím systému ticketů, kde bude zobrazena časová osa jednotlivých kroků řízení včetně vyjádření ze strany administrátora. Administrátorská část aplikace umožní pracovníkům domova mládeže spravovat přijaté žádosti, komunikovat se zájemci a vyhodnocovat je také na základě dojezdových vzdáleností či dalších relevantních kritérií.

Aplikace bude vyvíjena s využitím frameworku Next.js v jazyce TypeScript, pro tvorbu uživatelského rozhraní bude použit TailwindCSS a databázová vrstva bude řešena prostřednictvím PostgreSQL a ORM Prisma. Součástí práce bude také návrh databázového modelu a architektury aplikace, implementace uživatelských i administrátorských modulů a ošetření základních bezpečnostních aspektů.

Významnou součástí projektu bude nasazení hotové aplikace do produkčního prostředí na servery Střední průmyslové školy Třebíč, její praktické ověření v reálném provozu a zpracování uživatelské i technické dokumentace, která bude sloužit pro následnou údržbu a využívání systému v praxi.

Termín odevzdání: **29. března 2026, 23.00**

Vedoucí projektu: **Mgr. Matěj Brožek**

Oponent: **Ing. Ladislav Havlát**

Schválil: **Ing. Petra Hrbáčková, ředitelka školy**

ABSTRAKT

Maturitní práce má za úkol usnadnit proces přihlašování žáků a správu přihlášek vychovateli vytvořením aplikace pro správu přihlašovacího procesu. Úvod krátce popisuje aktuální řešení a problémy s ním spojené. Problematika stávajícího řešení popisuje důvody nutnosti vytvoření nové aplikace a zmiňuje všechny typy řešení, které byly zvažovány pro použití. Stanovení požadavků na novou aplikaci popisuje funkce, které by měla aplikace obsahovat. Architektura webové aplikace popisuje architekturální vzory, které byly použity při návrhu a vývoji aplikace a další teoretické informace. Technologie použité při vývoji popisují technologie, které byly použity při vývoji a důvody, proč byly zvoleny. V kapitole o implementaci webové aplikace je popsána samotná aplikace a její funkce. Poslední kapitola, vývoj a nasazení, popisuje proces vývoje a nasazení aplikace na produkční prostředí.

KLÍČOVÁ SLOVA

přihlašovací formulář, domov mládeže, webová aplikace, React, Next.js

ABSTRACT

The graduation thesis aims to streamline the student registration process and application management for educators by developing a dedicated management application. The introduction briefly describes the current solution and its associated issues. „The problem statement“ section outlines the reasons why a new application is necessary and mentions all types of solutions that were considered. Requirements specification for the new application describes the features it should include. The web application architecture section details the architectural patterns used during design and development, along with other theoretical information. „Used technologies“ describes the specific tools used during development and the reasons behind their selection. In the implementation chapter, the application itself and its functions are described. The final chapter, development and deployment, covers the development process and the transition of the application to a production environment.

KEYWORDS

application form, dormitory, web application, React, Next.js

PODĚKOVÁNÍ

Děkuji vedoucímu práce Mgr. Matěji Brožkovi za cenné rady a odborné vedení při zpracování této práce. Chtěl bych také poděkovat své rodině a blízkým přátelům za jejich podporu během celého období tvorby této práce. V neposlední řadě bych chtěl poděkovat virtuální zpěvačce Hatsune Miku za podporu, kterou mi dodala svojí pozitivní energií.

V Třebíči dne 25. 3. 2026

Podpis autora

PROHLÁŠENÍ

Prohlašuji, že jsem tuto práci vypracoval/a samostatně a uvedl/a v ní všechny prameny, literaturu a ostatní zdroje, které jsem použil/a. Při přípravě této práce jsem použil ChatGPT (https://chat.openai.com/) a GitHub Copilot (https://github.com/features/copilot) za účelem kontroly kódu, překladů v aplikaci a hledání chyb. Po použití těchto nástrojů jsem provedl kontrolu obsahu a přebírám za něj plnou odpovědnost.

V Třebíči dne 25. 3. 2026

Podpis autora

Obsah

Úvod **8**
1 Problematika stávajícího řešení **9**
1.1 Fyzické přihlášky **9**
1.2 Google Forms **9**
1.3 Externí řešení **10**
2 Stanovení požadavků na novou aplikaci **11**
2.1 Uživatelské funkce **11**
2.2 Administrátorské funkce **11**
3 Architektura webové aplikace **13**
3.1 Architekturální vzory **13**
3.2 Klient-Server **13**
3.2.1 Klient 14
3.2.2 Server 14
3.2.3 Komunikace mezi klientem a serverem 14
3.3 Komponentová architektura **15**
3.4 Autentizace a autorizace **16**
3.4.1 JWT (JSON Web Tokens) 16
3.4.2 Session-based autentizace 17
4 Technologie použité při vývoji **19**
4.1 TypeScript **19**
4.1.1 Typy a rozhraní 19
4.1.2 Transpilace 20
4.1.3 Standard ECMAScript 21
4.2 Next.js **21**
4.2.1 React 21
4.2.2 Správa stavu 22
4.2.3 React Server Components 24
4.2.4 Server Actions v Next.js 26
4.2.5 Kompilace Next.js 27
4.3 Prisma ORM **27**
4.3.1 PostgreSQL 28
4.4 TailwindCSS **29**

5 Implementace webové aplikace **30**
5.1 Návrh databáze a datového modelu **30**
5.2 Kontrola a validace dat **32**
5.3 Přihlášovací formulář a autentizace **32**
5.3.1 Anonymní odesílání přihlášek 33
5.3.2 Autentizace a autorizace pomocí knihovny Better Auth 33
5.3.3 Middleware pro ochranu veřejné API 34
5.4 Rozhraní pro žadatele **34**
5.4.1 Boční navigační menu 34
5.4.2 Sekce „Moje přihlášky“ 34
5.4.3 Formulář pro vytvoření přihlášky 35
5.4.4 Nastavení profilu 36
5.5 Rozhraní pro vychovatele **36**
5.5.1 Úvodní přehled 36
5.5.2 Nastavení chování aplikace 37
5.5.3 Zobrazení přijatých přihlášek 39
5.5.4 Detaily přihlášky 40
5.5.5 Export přihlášek do PDF 41
5.5.6 Archivace 42
5.5.7 Evidenční čísla 42
5.5.8 Massmail 43
5.5.9 Bodování přihlášek 43
6 Vývoj a nasazení **44**
6.1 Vývojové nástroje **45**
6.1.1 Docker 45
6.1.2 Užití statických analýzátorů kódu 45
6.1.3 Aktivní monitorování 47
6.2 Plánování vývoje pomocí GitHub Projects **48**
6.3 Verzovací systém Git **49**
6.4 GitHub Actions **49**
6.5 Nasazení na produkční server **50**
6.5.1 Inicializace aplikace 51
6.5.2 Architektura produkčního serveru 51
6.6 Praktické testování **52**

Závěr **53**
Seznam použité literatury **54**
Seznam obrázků **57**
Seznam tabulek **58**
Seznam výpisů **59**
Seznam zkratek a symbolů **60**

Úvod

Správa přihlašovacích formulářů na domovech mládeže a internátních zařízeních je často náročným a časově intenzivním procesem, který vyžaduje efektivitu při zpracování žádosti a případné komunikaci se žadateli. Cílem této práce je proto navrhnout a implementovat webovou aplikaci za použití moderních technologií, která tento proces zjednoduší a zpřehlední jej jak pro žadatele, tak pro vychovatele domova mládeže.

Historicky bylo přihlašování do domova mládeže řešeno prostřednictvím papírových formulářů, které byly vyplňovány ručně a posílány e-mailem. V posledních letech se však začaly hledat alternativy, které by umožnily digitalizaci tohoto procesu. Minulý rok bylo pilotně zavedeno zasílání přihlášek prostřednictvím Google Forms, což přineslo určité zlepšení. Nicméně tento systém má své limity, zejména pokud jde o evidenci a archivování přijatých žádostí. Přesně tyto problémy se snaží tato práce řešit vytvořením specializované webové aplikace.

Mezi primární funkce aplikace patří možnost vytváření a správy přihlášek, přičemž žadatelé budou moci sledovat stav své žádosti v reálném čase. Vychovatelé v domově mládeže pak získají nástroje pro správu přihlášek, automatizovanou komunikaci se žadateli a přehled o všech přijatých žádostech. Aplikace bude také obsahovat funkce pro hodnocení a výběr žadatelů na základě bodů, které budou automaticky uděleny dle uvedených odpovědí na otázky.

Výsledná aplikace by měla být přínosná pro všechny strany. Jmenovitě pak pro žadatele, kteří získají pohodlnou a transparentní cestu k podání přihlášky, a pro vychovatele, kteří budou mít efektivní nástroj pro správu a zpracování přihlášek.

1 Problematika stávajícího řešení

Jak již bylo zmíněno v úvodu, aktuální stav systému pro přihlašování a správu přihlášek pro domov mládeže je neoptimální a obsahuje množství funkcí, které lze implementovat pro celkové zlepšení uživatelské přívětivosti a pohodlnosti. Pro jednoduché ustanovení těchto funkcí je však potřeba podívat se na všechny typy, které kdy byly zvažovány pro použití, a následně tato data zohlednit při návrhu nového systému.

1.1 Fyzické přihlášky

Při analýze stávajících mechanismů podávání přihlášek se fyzická (papírová) forma jeví jako procesně nejméně efektivní varianta. Její limity však zároveň definují požadavky na modernizaci a digitalizaci celého systému. Mezi kritické nedostatky, které je nutné v rámci návrhu nového řešení eliminovat nebo je omezit, patří zejména:

- Nutnost fyzického doručení přihlášky na určené místo. To může být pro některé žadatele komplikované.
- Obtížná správa a archivace fyzických přihlášek. Papírové dokumenty se těžce hledají, třídí a uchovávají, což dokáže zvýšit administrativní zátěž pro vychovatele.
- Omezené možnosti pro automatizaci procesu hodnocení a kalkulace bodů na základě odpovědí žadatelů. Nutnost manuálního zpracování vede k chybovosti a daleko vyšší časové náročnosti.

1.2 Google Forms

Google Forms je online nástroj pro tvorbu formulářů, sběr a statistiku dat. Nabízí široké možnosti přizpůsobení formulářů, integraci s dalšími službami Google a automatické shromažďování odpovědí do přehledných tabulek [1].

Mezi hlavní výhody Google Forms patří hlavně intuitivní uživatelské rozhraní, jednoduché nastavení a možnost rychlého sdílení formulářů prostřednictvím odkazu. Dále nabízí automatický export dat do tabulek. Všechny tyto funkce výrazně usnadňují přihlašovací proces, avšak existují i funkce, které jsou zásadní a chybí. Pro školní rok 2025/2026 byly Google Forms pilotně využity jako nástroj pro sběr přihlášek. Část procesu přihlašování z administrativní strany tvoří např. archivace přijatých přihlášek

ve formátu PDF[1], či komunikace se žadateli. Tyto funkce však Google Forms nenabízí, což vede k nutnosti manuálního zpracování.

1.3 Externí řešení

Externí řešení patří zdaleka k nejvhodnějším a nejflexibilnějším možnostem, jak řešit přihlašovací proces. Mezi hlavní výhody patří především fakt, že produkty lze přizpůsobit na míru dle požadavků a potřeb domova mládeže. Nabízejí též opravu chyb, aktualizace a technickou podporu, což může být velice užitečné pro zajištění hladkého chodu systému. Mezi nevýhody patří především finanční náročnost, jelikož externí řešení často vyžadují měsíční nebo roční poplatky za podporu a údržbu.

Vlastnost	Fyzické přihlášky	Google Forms	Externí řešení	Studentský projekt
Snadná manipulace s přihláškou	ne	ano	ano	ano
Automatizace hodnocení přihlášek	ne	částečně	ano	ano
Archivace přihlášek	ne	částečně	ano	ano
Komunikace se žadateli	ne	ne	možná	ano
Implementace na míru	ne	ne	ano	ano
Možnost expanze funkcí	částečně	ne	ano	ano
Náklady	nízké	nízké	vysoké	nízké

Tabulka 1: Zjednodušené porovnání řešení přihlašovacího procesu do domova mládeže

[1]Tento krok byl řešen automaticky pomocí zautomatizovaného skriptu v Google Sheets, z vlastní zkušenosti však bylo toto řešení velice chybové a často se muselo upravovat.

2 Stanovení požadavků na novou aplikaci

Při diskuzi o tvorbě nové aplikace byly též stanoveny požadavky na funkce, které by měla aplikace obsahovat. Vycházeli jsme především z problémů, které přinášela stávající řešení. Tyto požadavky můžeme pro přehlednost rozdělit do dvou hlavních kategorií: **požadavky na uživatelské funkce** a **požadavky na funkce administrátorské**. Na základě těchto požadavků byly následně vybrány technologie užité pro vývoj aplikace.

2.1 Uživatelské funkce

Uživatelskými funkcemi se rozumí funkce, které jsou přístupné a využívané samotnými žadateli o ubytování. Mezi některé z těchto funkcí patří:

- Zobrazení přehledného formuláře pro podání přihlášky, který bude obsahovat všechny potřebné informace a otázky pro správné vyplnění žádosti. Zároveň bude obsahovat validaci pro zajištění správnosti a úplnosti zadaných dat společně s kolonkami pro nahrání potřebných souborů.
- Možnost sledování přihlášky, včetně jejího stavu, v reálném čase. Žadatelé budou mít jednoduchý přehled o tom, v jaké fázi se jejich žádost nachází, a budou informováni o případných změnách stavu.
- Využití stejného účtu pro podání a správu více přihlášek (předpokládá se, že uživatelé budou podávat přihlášky pro více svých dětí a ve více letech).

2.2 Administrátorské funkce

Do administrátorských funkcí spadají funkce, které jsou standardně přístupné pouze pro vychovatele a pracovníky domova mládeže, kteří mají na starosti správu přihlášek.

- Možnost spravovat přijaté žádosti, včetně možnosti prohlížet a upravovat informace o žadatelích, měnit stav žádostí a přidávat poznámky k jednotlivým žádostem.
- Automatizace procesu hodnocení a kalkulace bodů na základě odpovědí žadatelů. Aplikace bude automaticky udělovat body za jednotlivé odpovědi na otázky v přihlášce, což usnadní a zrychlí proces hodnocení žádostí.
- Možnost komunikace se žadateli přímo z aplikace, a to ať už prostřednictvím e--mailu, tak i za pomoci funkce pro hromadnou korespondenci, která využije interního školního e-mailového serveru pro odesílání zpráv.

- Zabezpečení přístupu k administrátorským funkcím pomocí autentizačního systému s RBAC[2], který zajistí různé úrovně přístupu pro různé role vychovatelů.
- Automatická generace evidenčních (jednacích) čísel, která budou přidělována každé přihlášce.
- Možnost exportu dat do tabulkového a PDF formátu pro potřeby archivace a exportu do externích systémů.

[2]Role-Based Access Control je systém pro efektivní správu přístupu k zabezpečeným informacím pomocí rolí a oprávnění [2].

3 Architektura webové aplikace

Vývoj jakékoliv aplikace začíná návrhem její architektury. Ta dokáže přibližně nastínit, jak do sebe budou jednotlivé části aplikace zapadat a také se od ní odvíjí výběr technologií, které budou při vývoji použity.

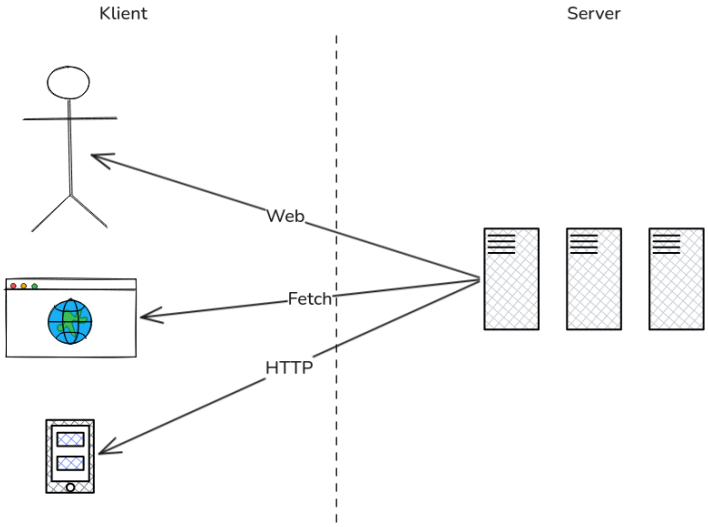
3.1 Architekturální vzory

Softwarový architekturální vzor je řešení, které se opakovaně používá v rámci vývoje softwaru [3]. Mezi vzory, se kterými se lze běžně setkat, patří například Model-View-Controller (MVC), Model-View-ViewModel (MVVM) a Client-Server. Architekturální vzory poskytují jak jasnou strukturu organizace kódu, tak i jasně definované role pro jednotlivé části aplikace, což výrazně usnadňuje vývoj, údržbu a rozšiřování aplikace.

3.2 Klient-Server

Klient-server architektura je jedním z nejběžnějších a nejpoužívanějších architekturálních vzorů pro vývoj webových aplikací. Tento vzor se zakládá na rozdělení aplikace do dvou hlavních částí: klienta a serveru.

V kontextu webového vývoje se fungování moderních webových aplikací opírá o rozdělení odpovědnosti mezi uživatelské rozhraní (*client-side*) a aplikační logiku běžící na serveru (*server-side*). Obecně lze říci, že klientská část je odpovědná za zobrazení a prezentaci dat, zatímco server je odpovědný za poskytování a zpracování samotných dat.



Obrázek 1: Vizualizace client-server architektury

3.2.1 Klient

Klientská část aplikace, někdy nazývaná také *front-end*, představuje kód a soubory, které jsou přeneseny do prohlížeče uživatele a vykonávány přímo na jeho zařízení. Hlavním účelem těchto dat je vykreslit uživatelské rozhraní, zpracovávat interakce s cílovým uživatelem a zpětně komunikovat se serverovou částí aplikace.

3.2.2 Server

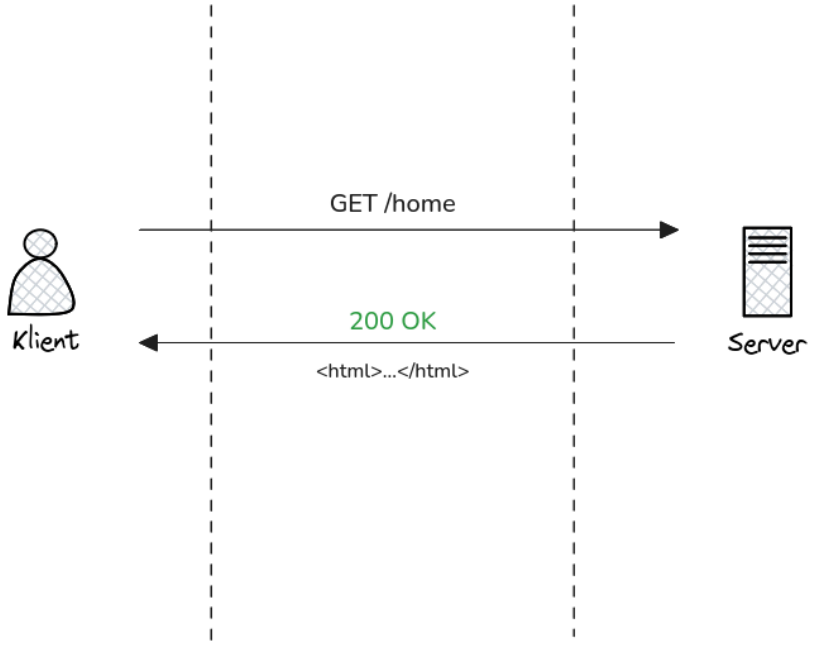
Serverová část aplikace, také označována jako *back-end* je část aplikace, která běží na vzdáleném zařízení (server), není tedy přístupná přímo uživateli. Serverová část aplikace je nejčastěji zodpovědná za interní zpracování dat, komunikaci s databází, autentizaci uživatelů a poskytování dat klientovi (včetně samotné webové stránky). Obecně by se tedy dalo říci, že tato část aplikace provádí úkony, které jsou považovány za nebezpečné (např. zápis do databáze) nebo neověřitelné (např. validace dat) při vykonávání na straně klienta.

3.2.3 Komunikace mezi klientem a serverem

Komunikace mezi klientem a serverem na aplikační vrstvě může používat různé protokoly, mezi nejběžněji používané protokoly patří HTTP (HyperText Transfer Protocol), WebSocket a GraphQL. V této práci probíhá komunikace mezi klientem a serverem primárně pomocí protokolu HTTP.

Protokol HTTP dělí komunikaci na dvě hlavní části: požadavek (*request*) a odpověď (*response*). Klient (prohlížeč) odešle požadavek na server, který tento požadavek zpracuje a následně odešle zpět odpověď. Požadavek i odpověď obsahují různé informace.

HTTP požadavek je definován především metodou (např. GET pro čtení či POST pro zápis), která určuje typ operace, a cílovou adresou URL identifikující konkrétní zdroj. Nedílnou součást tvoří hlavičky, nesoucí metadata typu autentizačních údajů či formátu dat, a volitelně také tělo požadavku obsahující samotná data k odeslání. Server následně na tento podnět reaguje strukturou, jejímž klíčovým prvkem je stavový kód (např. 200 OK či 404 Not Found), ten poskytuje okamžitou informaci o výsledku zpracování. Obdobně jako u požadavku, i odpověď obsahuje specifické hlavičky a zpravidla i tělo, které nese klientem vyžádaný obsah, nejčastěji ve formátu JSON nebo HTML [4].



Obrázek 2: Vizualizace komunikace mezi klientem a serverem pomocí HTTP protokolu

3.3 Komponentová architektura

Komponentová architektura (*Component-based Architecture*) je architektura, která je hojně užívaná v případě knihoven jako je React. Tento přístup se zaměřuje na rozdělení uživatelského rozhraní do menších, znovupoužitelných částí, které jsou nazývány komponentami. Každá komponenta je samostatná jednotka, která obsahuje jak logiku, tak i prezentaci určité části uživatelského rozhraní. Komponenty mohou být

hierarchicky uspořádány, což umožňuje vytváření složitějších rozhraní z jednodušších stavebních bloků [3].

3.4 Autentizace a autorizace

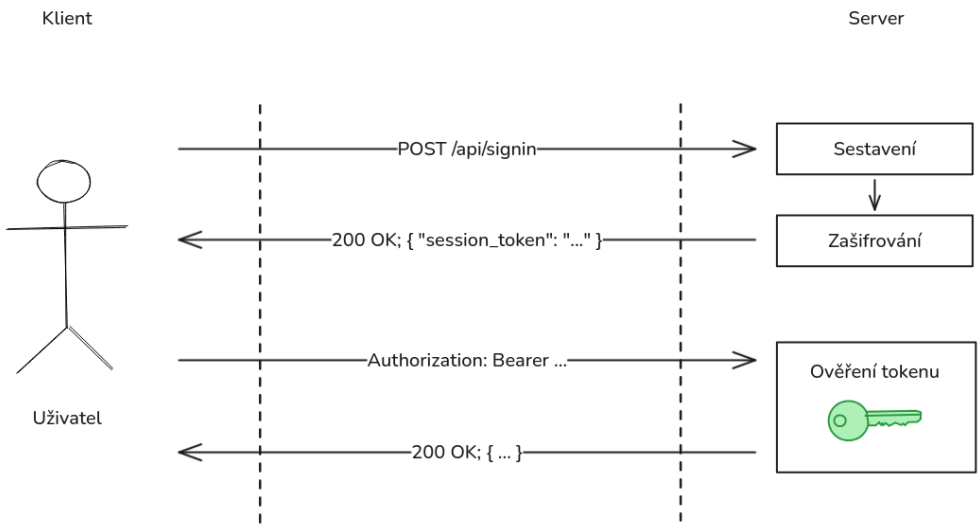
Pro zabezpečení přístupu k funkcím, jež jsou určeny pouze pro oprávněné klienty (uživatele), je potřebná implementace systému pro autentizaci a autorizaci. Autentizace je proces ověření identity uživatele, zatímco autorizace určuje, jaké akce může autentizovaný uživatel provádět.

V dnešní době dělíme autentizaci na dva primární typy – **Session-based authentication** a **JWT (JSON Web Tokens)**. Často se však můžeme setkat i s termíny jako jsou **stateful** a **stateless authentication**; tyto termíny přímo popisují, zda server uchovává stav o přihlášeném uživateli (stateful), nebo nikoliv (stateless).

3.4.1 JWT (JSON Web Tokens)

JWT je standard pro bezpečnou **stateless** autentizaci (tj. neuchovává stav přihlášeného uživatele na serveru). Autentizace je rozdělena na dva hlavní kroky – přihlášení a ověření tokenu. Při přihlášení klient vyšle požadavek na server s přihlašovacími údaji (např. uživatelské jméno a heslo). Server ověří tyto údaje a pokud jsou správné, vygeneruje JWT token (obsahující informace o uživateli a jeho oprávněních ve formátu JSON), tento token je zašifrován pomocí asymetrického klíče a následně odeslán zpět klientovi. Klient si tento token uloží (např. do localStorage nebo cookies) a při každém dalším požadavku na server ho přiloží v hlavičce `Authorization`. Server následně ověří platnost tokenu (např. kontrolou podpisu a expirace) a pokud je token platný, povolí přístup k požadovaným zdrojům [5].

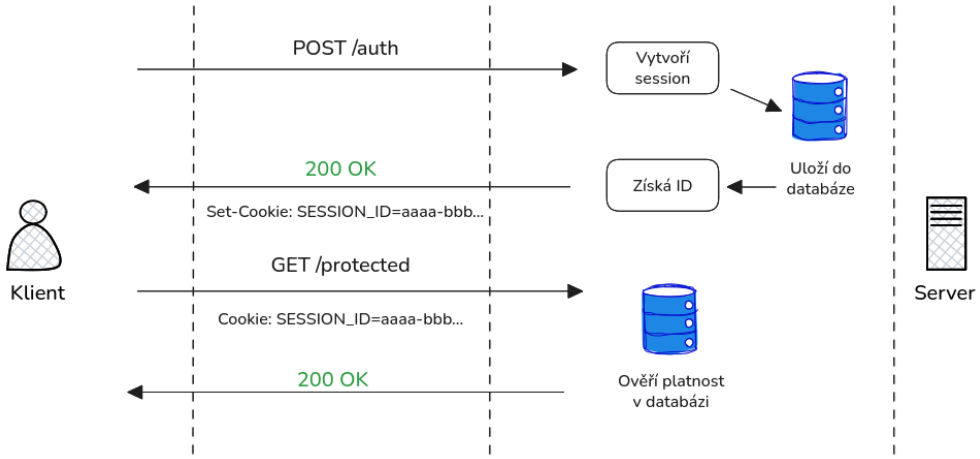
Oproti session-based autentizaci má tento způsob hlavní nevýhodu v tom, že server nemá možnost uživatele odhlásit před vypršením platnosti tokenu, jelikož server neuchovává žádný stav o přihlášeném uživateli. Této funkce lze však dosáhnout implementací blacklistu neplatných tokenů na straně serveru, v takovém případě však JWT ztrácí svůj hlavní benefit – stateless charakter [6].



Obrázek 3: Vizualizace JWT autentizace

3.4.2 Session-based autentizace

Druhým hojně užívaným způsobem pro autentizaci je tzv. session-based autentizace. Tento styl je oproti JWT **stateful**, ukládá tedy stav přihlášeného uživatele na serveru (v databázi). Při přihlášení klient odešle požadavek na server s přihlašovacími údaji. Server ověří tyto údaje a pokud jsou správné, vytvoří novou session (relaci) pro uživatele a vygeneruje unikátní identifikátor session (session ID). Tento identifikátor je následně odeslán zpět klientovi, který si ho uloží do cookies. Při každém dalším požadavku na server klient automaticky přiloží cookies obsahující session ID. Server následně ověří platnost session ID (např. kontrolou, zda session stále existuje v databázi) a pokud je platné, povolí přístup k požadovaným zdrojům [6].



Obrázek 4: Vizualizace session-based autentizace

Vlastnost	JWT Token	Session
Uchovávání stavu na serveru	ne	ano
Možnost odhlášení uživatele před vypršením platnosti	složitější	ano
Škálovatelnost	vysoká	nízká

Tabulka 2: Porovnání standardů pro autentizaci a autorizaci

Důležité je zmínit i zabezpečení těchto autentizačních mechanismů. Oba způsoby vyžadují uložení dat na straně klienta (JWT token nebo session ID), což dokáže být jednoduše zneužito. Pro zabezpečení těchto dat je důležité použít bezpečné úložiště (např. Cookies s parametry `Secure`, `HttpOnly`), které zabrání čtení cookies přímo z klientského JavaScriptu a implementovat další bezpečnostní opatření, jako je například ochrana proti CSRF útokům, použití HTTPS pro šifrování komunikace a pravidelná rotace klíčů pro JWT tokeny (což nadále komplikuji implementaci JWT autentizace) [7].

V obecné rovině lze říct, že zatímco JWT představuje průmyslový standard pro autorizaci v rámci distribuovaných systémů (např. aplikace s více instancemi či aplikaci využívající mikroslužby) a API, session-based přístup zůstává optimálním řešením pro monolitické webové aplikace, a to především díky vyšší bezpečnosti při ukládání citlivých údajů a jednodušší invalidaci aktivních relací.

4 Technologie použité při vývoji

Při vývoji moderní webové aplikace je klíčové zvolit technologie, které kromě splnění požadavků na funkcionalitu zajistí i dlouhodobou udržitelnost, škálovatelnost a bezpečnost aplikace. Následující technologie byly vybrány na základě jejich výhod a existujících zkušeností.

4.1 TypeScript

TypeScript je staticky typované rozšíření JavaScriptu, které přidává podporu pro typy, třídy, rozhraní a další funkce, které lze běžně najít ve striktně staticky a objektově orientovaných jazycích. Přidává silnou typovou kontrolu, která se prosazuje již při samotné transpilaci kódu do JavaScriptu, což pomáhá odhalit chyby už během vývoje a zvyšuje kvalitu kódu [8].

TypeScript byl zvolen pro tento projekt z několika důvodů. Prvním z nich je fakt, že je hojně využíván v moderním webovém vývoji, což zajišťuje širokou podporu a množství knihoven a nástrojů, které jsou s ním kompatibilní [9]. Dále přináší výhody v podobě lepší čitelnosti a údržby kódu, což je klíčové pro dlouhodobou udržitelnost projektu. TypeScript také umožňuje využívat pokročilé funkce a syntaxi (např. generika), které nejsou nativně podporovány v JavaScriptu, což může zefektivnit vývoj a zlepšit celkovou strukturu kódu.

4.1.1 Typy a rozhraní

Jak již bylo na začátku kapitoly zmíněno, TypeScript přináší do JavaScriptu rozsáhlou podporu typů. Typy jsou definovány pomocí klíčového slova `type` a umožňují vývojářům definovat vlastní datové struktury, které mohou být použity k zajištění správnosti dat v aplikaci. Mezi vybrané základní předdefinované datové typy pak patří:

- `string` pro textové řetězce
- `number` pro číselné hodnoty
- `boolean` pro logické hodnoty (true/false)
- `array` pro pole hodnot
- `enum` pro výčtové typy
- `any` pro hodnoty, které mohou být jakéhokoliv typu[3]
- `void` pro funkce, které nevracejí žádnou hodnotu

- `null` a `undefined` pro reprezentaci neexistujících nebo nedefinovaných hodnot

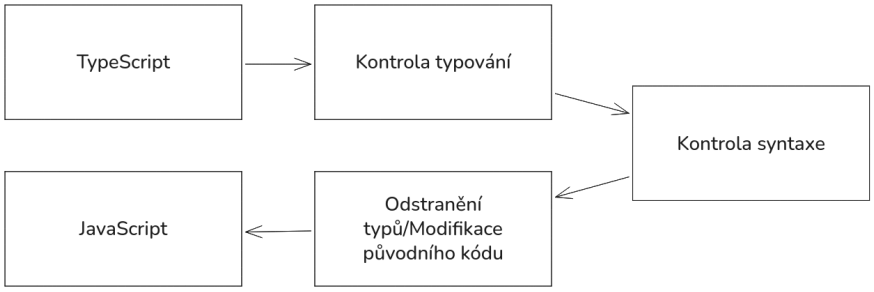
Pomocí TypeScriptu lze však specifikovat i složitější datové struktury. Typy lze kombinovat pomocí logických operátorů `&` (průnik typů) a `|` (sjednocení typů). Komplexní datové struktury lze pak definovat pomocí rozhraní (`interface`) či `type` aliasů, které umožňují definovat strukturu objektů a jejich vlastností.

Výpis 1: Ukázka práce s rozhraními – definice rozhraní a typu odkazujícího na vlastnost rozhraní

```
1 interface User {
2   id: number;
3   name: string;
4 }
5 type UserId = User["id"]; // number
```

4.1.2 Transpilace

Pojmem *transpilace* se rozumí proces převodu kódu z jednoho programovacího jazyka do druhého. V případě TypeScriptu se jedná o převod kódu napsaného v TypeScriptu zpět do běžného JavaScriptu, který lze následně vykonávat nativně v prohlížeči či v serverovém prostředí (Node.js, Bun…). Transpilace umožňuje vývojářům využívat pokročilé funkce a syntaxi TypeScriptu, zatímco výsledný kód zůstává kompatibilní s širokou škálou prostředí, která podporují JavaScript.



Obrázek 5: Vizualizace procesu transpilace TypeScriptu do JavaScriptu

[3]Použití typu `any` se nedoporučuje [10]. Jeho použití vede k vypnutí statické kontroly typů pro danou proměnnou, což může vést k chybám, které by jinak mohly být odhaleny během vývoje.

4.1.3 Standard ECMAScript

Vývoj syntaxe a funkcí v JavaScriptu (a tedy i TypeScriptu) je řízen standardem ECMAScript, který je pravidelně aktualizován a přináší modernizace syntaxe a funkcionalit do JavaScriptu [11]. TypeScript nabízí podporu pro verzi ECMAScript standardu až do ES5 [12]. Mezi moderní funkce, které standard ECMAScript přinesl v posledních letech, patří zejména šipkové funkce, třídy, moduly a podpora asynchronního programování [13].

Funkce	TypeScript	JavaScript
Statické typování	ano	ne
Podpora pro rozhraní a typy	ano	ne
Podpora pro moderní syntaxi	ano	částečně
Zachycení chyb během vývoje	ano	ne
Kompatibilita s prohlížeči	ne	ano
Počáteční konfigurace	z počátku	žádné
Hromadné změny kódu	ano	složitější

Tabulka 3: Porovnání vybraných aspektů skriptovacích jazyků a srovnání TypeScriptu a JavaScriptu

4.2 Next.js

Next.js je webový framework, který je postaven na Reactu a umožňuje tvorbu kompletních webových aplikací s podporou pokročilých funkcí, jako je *Server-Side Rendering* (SSR), nebo *Server Actions* [14].

4.2.1 React

React je knihovna pro tvorbu uživatelských rozhraní, která umožňuje vytváření komponent založených na stavech a vlastnostech přímo v JavaScriptu či TypeScriptu [15]. Jedná se o jeden z nejpoužívanějších nástrojů pro vývoj webových aplikací. Díky přímé integraci v Next.js umožňuje efektivní tvorbu dynamických a interaktivních uživatelských rozhraní.

React umožňuje tvorbu *znovupoužitelných komponent* (viz Kapitola 3.3). Tyto komponenty jsou prosté funkce nebo třídy[4], které přijímají vstupní data (*props*, též známé v HTML jako atributy). Komponenty mohou také spravovat svůj vlastní stav – *state*, což umožňuje vytváření interaktivních prvků uživatelského rozhraní.

Každý soubor s příponou `.tsx` nebo `.jsx` představuje soubor podporující speciální syntaxi JSX, ta umožňuje kombinovat kód podobný HTML přímo do JavaScriptu/ TypeScriptu. Tento kód je následně přeložen do nativního JavaScriptu, který je vykonáván v prohlížeči.

Výpis 2: Ukázka komponenty v Reactu

```
1   function HelloWorld() {
2     const [greeting, setGreeting] = React.useState("Ahoj, Světe!");
3
4     return (
5       <div>
6         <h1>{greeting}</h1>
7         <button onClick={() => setGreeting("Ahoj, Next.js!")}>
8           Změnit pozdrav
9         </button>
10      </div>
11    );
12  }
```

4.2.2 Správa stavu

Správa stavu (*State Management*) je jedním z klíčových aspektů knihovny React. Stav představuje data v paměti, která se mohou měnit v průběhu života komponenty a ovlivňovat její chování a vzhled. React nejčastěji využívá pro správu stavu vestavěný hook[5] `useState`, který umožňuje definovat a aktualizovat stav na úrovni funkční komponenty.

[4] V moderních verzích knihovny React je doporučeno používat výhradně funkční komponenty.

[5] Hook je prostá JavaScriptová funkce, která začíná na „use“. Používají se pro přidání logiky do funkčních komponent.

Pro zamezení tzv. *prop-drillingu* (situace, kdy je potřeba předávat stavová data z rodičovské komponenty do hluboko zanořených komponent přes několik úrovní atributů komponent), lze využít kontextu (*Context API*), který umožňuje sdílet data mezi komponentami bez nutnosti předávat je přes každou úroveň komponent.

Výpis 3: Ukázka využití Context API pro správu stavu v Reactu

```
1  // Vytvoření kontextu pro sdílení stavu mezi komponentami
2  const Context = React.createContext();
3  function KomponentaA() {
4    // Definice stavu, který bude sdílen mezi komponentami
5    const [stav, setStav] = React.useState("Nějaký stav");
6
7    return (
8      // Poskytnutí stavu a funkce pro jeho aktualizaci
       prostřednictvím kontextu
9      <Context.Provider value={{ stav, setStav }}>
10       <KomponentaB />
11     </Context.Provider>
12   );
13 }
14 function KomponentaB() {
15   // Přístup ke sdílenému stavu pomocí kontextu
16   const { stav, setStav } = React.useContext(Context);
17
18   return (
19     <div>
20       <p>Aktuální stav: {stav}</p>
21       <button onClick={() => setStav("Nový stav")}>
22         Změnit stav
23       </button>
24     </div>
25   );
26 }
```

V případě složitějších aplikací, které vyžadují komplexnější správu stavu, lze využít některé z externích knihoven. V tomto projektu byla zvolena knihovna Jotai, která nabízí jednoduché řešení pro správu stavu pomocí primitivních jednotek v podobě *atomů* [16].

Výpis 4: Ukázka využití knihovny Jotai pro správu stavu

```
1 const counterAtom = atom(0);
2 function Counter() {
3   const [count, setCount] = useAtom(counterAtom);
4
5   return (
6     <p>Počet: {count}</p>
7   );
8 }
```

4.2.3 React Server Components

React Server Components (RSC) je speciální typ komponenty v Reactu, která umožňuje vykonávání kódu komponenty na serveru místo v prohlížeči. V Next.js lze rozlišit RSC a běžné komponenty na straně klienta pomocí direktivy `"use client"` umístěné na začátku souboru. Pokud tato direktiva chybí, React automaticky považuje všechny komponenty definované v daném souboru za serverové komponenty [17].

Tento speciální typ komponenty umožňuje vývojářům přistupovat ke zdrojům na serveru, jako je databáze nebo souborový systém přímo z komponenty, aniž by bylo nutné vytvářet API rozhraní pro komunikaci. Jedním z úskalí RSC je, že tyto komponenty nemohou používat interaktivní prvky (např. `onClick` události nebo `useState` hook).

Využití samotné RSC také prodlužuje dobu načítání stránky, protože React čeká na dokončení obsluhy serverové komponenty před tím, než odešle výsledná data do prohlížeče klientovi, nelze tedy použít reaktivní funkce pro kontrolu načtení obsahu. Pro zvýšení uživatelské přívětivosti (UX) existuje proto tzv. *Suspense* komponenta, ta dokáže zobrazit náhradní obsah (např. indikátor načítání) zatímco server čeká na dokončení vykonání RSC.

Výpis 5: Ukázka React Server Component a jejího použití s Suspense

```
1   // server-component.tsx
2   import React from "react";
3
4   async function ServerComponent() {
5     const data = await fetchDataFromDatabase();
6
7     return (
8       <div>
9         <h1>Data ze serveru:</h1>
10        <pre>{JSON.stringify(data, null, 2)}</pre>
11      </div>
12    );
13  }
14
15  function Page() {
16    return (
17      <div>
18        <h1>Moje stránka s RSC</h1>
19        <React.Suspense fallback={
20          <div>Načítání dat ze serveru...</div>
21        }>
22          <ServerComponent />
23        </React.Suspense>
24      </div>
25    )
26  }
```

RSC lze do jisté míry přirovnat ke klasickému PHP, které kód vykonává na serveru a uživateli pošle až hotovou HTML stránku. Oproti PHP však RSC umožňuje kombinovat serverový a klientský kód v rámci jedné aplikace a za pomocí komponentové architektury, což přináší větší flexibilitu a možnosti pro vývojáře.

4.2.4 Server Actions v Next.js

Server Actions (česky Serverové akce nebo Funkce na straně serveru) nahrazují potřebu vytváření samostatné API na serveru, kterou by bylo nutné z klientské strany volat. Místo toho lze funkce, jež jsou definovány ve speciálním souboru volat přímo z komponent na straně klienta. Next.js interně automaticky vytvoří potřebné API na pozadí [18].

Jako příklad můžeme vytvořit jednoduchou funkci, jejíž úkolem bude vrátit aktuální čas ze serveru. Server Actions jsou definovány v souborech, které začínají direktivou `"use server"`.

Výpis 6: Ukázka Server Action v Next.js

```
1 "use server";
2
3 export async function getServerTime() {
4   return new Date().toISOString();
5 }
```

Funkci lze následně importovat a volat přímo z komponenty na straně klienta.

Výpis 7: Ukázka komponenty v Next.js využívající Server Action

```
1  "use client";
2  export function TestovaciKomponenta() {
3    const [cas, setCas] = useState(null);
4
5    useEffect(() => {
6      getServerTime().then(setCas);
7    }, [getServerTime]);
8
9    return <div>Aktuální čas ze serveru: {cas ?? "Načítání..."}</
     div>;
10 }
```

4.2.5 Kompilace Next.js

Pro optimalizaci výkonu a zajištění kompatibility se širokou škálou prohlížečů, framework Next.js využívá pro kompilaci kódu nástroj SWC a Turbopack. Při spuštění příkazu, který kompiluje a zajištuje přípravu kódu na produkčním prostředí (typicky `next build`), se odehraje několik zásadních kroků. Lze je pozorovat při samotném běhu tohoto příkazu (v terminálu se zobrazí informace o jednotlivých krocích kompilace):

1. Validace JavaScriptu a TypeScriptu pro zajištění správnosti kódu.
2. Transformace kódu do podoby optimalizované pro produkční prostředí (odstraňování mrtvého kódu, optimalizace importů, atd.).
3. Transpilace kódu z TypeScriptu do JavaScriptu. (TSC/Bun)
4. Spuštění předkompilovatelných funkcí (pre-rendering), jako jsou React Server Components a Server Actions. (Turbopack)
5. Vytvoření produkčního balíčku připraveného pro nasazení.

```
$ next build
▲ Next.js 16.1.7 (Turbopack)
- Environments: .env
- Experiments (use with caution):
  ✓ authInterrupts
  ✓ globalNotFound
  · serverActions

  Creating an optimized production build ...
✓ Compiled successfully in 21.5s
✓ Finished TypeScript in 14.7s
  Collecting page data using 15 workers  .
  Collecting page data using 15 workers  ..
  Collecting page data using 15 workers  ...
✓ Collecting page data using 15 workers in 3.2s
  Generating static pages using 15 workers (0/19)  [=   ]
  Generating static pages using 15 workers (7/19)  [  ==]
✓ Generating static pages using 15 workers (19/19) in 1372.8ms
✓ Finalizing page optimization in 8.3ms
```

Obrázek 6: Výstup příkazu `next build` zobrazující jednotlivé kroky kompilace a optimalizace kódu v Next.js

4.3 Prisma ORM

Prisma je moderní ORM (Object-Relational Mapping) nástroj pro TypeScript, který slouží k interakci s databází pomocí automaticky generovaného typovaného API [19]. Jedná se o jednu z nejpopulárnějších možností pro práci s databázemi v TypeScriptu,

a díky své jednoduchosti pro vykonávání jednoduchých CRUD operací byla ideální volbou pro tento projekt.

Každý projekt definuje své schéma databáze v souboru zakončeného příponou `.prisma`. Tento soubor obsahuje modely, které reprezentují tabulky a jejich vztahy v databázi. Prisma následně na základě tohoto schématu generuje typované API pro interakci s databází.

Výpis 8: Ukázka schématu databáze v Prisma ORM s modelem `User`

```
1 model User {
2   id       Int      @id @default(autoincrement())
3   email    String   @unique
4   name     String?
5 }
```

Výpis 9: Ukázka použití generovaného kódu pro práci s modelem `User`

```
1 const prisma = new PrismaClient();
2 await prisma.user.create({
3   data: {
4     email: "<email>",
5     name: "<name>"
6   }
7 });
```

Prisma podporuje širokou škálu databázových systémů, včetně PostgreSQL, MySQL, SQLite a dalších [19].

4.3.1 PostgreSQL

PostgreSQL je relační SQL databázový systém, který byl společně s Prismou zvolen jako hlavní databázové řešení pro tento projekt. Jedná se o jeden z nejpoužívanějších databázových systémů s pokročilými funkcemi, jako je podpora transakcí, bezpečnost pomocí Row-Level Security (RLS), pokročilé datové typy (JSON, UUID), či *Full-Text Search* (fulltextové vyhledávání). PostgreSQL také dodržuje takzvaný ACID standard, který definuje garance přístupu a platnost dat v případě chybových událostí, jako jsou výpadky proudu, náhlé chyby či jiné externí okolnosti, které by mohly ovlivnit chod databáze [20].

Velkou výhodou databázového systému PostgreSQL je pak jeho rozšiřitelnost, jeho funkcionalitu lze pak jednoduše expandovat pomocí rozšíření [20]. Mezi některá známá rozšíření pak patří například PgVector, který implementuje rozšířené vektorové vyhledávání, což je velkou výhodou pro optimalizované vyhledávání ve velkých datasetech [21].

4.4 TailwindCSS

TailwindCSS je podpůrný CSS framework pro moderní webový vývoj, který umožňuje rychlé a efektivní vytváření uživatelských rozhraní pomocí tříd s předdefinovanými styly [22]. Vývojářům je umožněno vytvářet responzivní a přizpůsobitelná rozhraní bez nutnosti psaní vlastního CSS kódu od nuly. Poskytuje širokou škálu tříd, které pokrývají různé aspekty stylování, jako je rozvržení, barvy, responzivita, typografie a další.

Výpis 10: Jednoduché stylování za použití frameworku TailwindCSS

```
1 <div class="flex items-center justify-center">
2   <h1 class="text-2xl font-bold font-[Poppins]">Test</h1>
3 </div>
```

Na první pohled TailwindCSS dokáže připomínat ostatní CSS frameworky, jako je například Bootstrap, jeho zásadní rozdíl však spočívá v granularitě (jemnosti) stylů [22]. Zatímco TailwindCSS abstrahuje jednotlivé styly do samotných CSS tříd, Bootstrap vytváří ucelené a předem definované styly (v angličtině *opinionated design*) pro celé komponenty.

Velkou výhodou TailwindCSS oproti ostatním CSS frameworkům je také *JIT Kompilátor*, který umožňuje optimalizaci balíčku stylů, který se v produkčním prostředí stahuje na klientské zařízení. Kompilátor funguje tak, že z balíčku vyřadí styly, které nejsou v projektu využity, a výsledná velikost balíčku zaslaného na klienta se tak dokáže dramaticky zmenšit [22].

5 Implementace webové aplikace

Největší část celého vývojového procesu tvoří samotný vývoj a programování funkcí. V této kapitole jsou popsány konkrétní prvky zohledněny při vývoji a jednotlivé funkce aplikace.

5.1 Návrh databáze a datového modelu

Jak již bylo zmíněno v kapitole o použitých technologiích, pro práci s databází bylo zvoleno ORM Prisma. Samotný databázový model pak vychází z výchozího modelu používaného knihovnou Better Auth (viz Kapitola 5.3). Tento model byl následně rozšířen o další sloupce a tabulky, které byly zapotřebí pro implementaci webové aplikace.

Mezi některé důležité tabulky v modelu patří:

- `user` – tabulka pro ukládání informací o uživatelích, jako je e-mail, role a další.
- `application` – tabulka pro ukládání informací o přihláškách, jako je stav přihlášky, datum podání, a další.
- `application_config` – tabulka pro ukládání konfiguračních nastavení aplikace, jako jsou stav přijímání přihlášek, datum uzávěrky apod.
- `person` – tabulka pro ukládání informací o osobách, které jsou spojeny s přihláškami (např. žadatel a zák. zástupce).

Obrázek 7: Databázový model aplikace jako Entity-Relationship Diagram

5.2 Kontrola a validace dat

Protože jsou data zadaná uživateli často náchylná k chybovosti, je důležité zabezpečit formuláře a API rozhraní kontrolou a validací vstupních dat. Validaci je důležité provádět duálně – jak na straně klienta (pro rychlou odezvu bez nutnosti komunikace se serverem), tak i na straně serveru (pro zajištění bezpečnosti a integrity dat). Důvodem pro dvojitý přístup k validaci je fakt, že validace na straně klienta může být snadno obejita (např. pomocí nástrojů pro vývojáře v prohlížeči), zatímco validace na straně serveru obejita být nemůže a zajišťuje správnost dat, která jsou uložena v databázi.

Pro práci s validací dat byla zvolena knihovna Zod. Pomocí Zodu lze jednoduše definovat schéma pro validaci dat a následně použít toto schéma pro validaci dat jak na straně klienta, tak i na straně serveru. Zod umožňuje definovat jak komplexní, tak jednoduché datové struktury, je jednoduše použitelný a nabízí širokou škálu vestavěných validací pro různé datové typy (např. e-mail, URL, čísla, atd.). Zod zároveň využívá výhod TypeScriptu a z modelu lze automaticky odvodit typy, což zajišťuje konzistenci mezi validací a typovou kontrolou v celé aplikaci [23].

Výpis 11: Příklad užití knihovny Zod

```
1 import { z } from "zod";
2
3 const userSchema = z.object({
4   email: z.email(),
5   password: z.string().min(8, "Minimálně 8 znaků!"),
6   name: z.string().optional(),
7 });
8 // TS typ odvozený ze schématu
9 type User = z.infer<typeof userSchema>;
```

5.3 Přihlášovací formulář a autentizace

Po úvodním otevření aplikace je uživatel automaticky přesměrován na přihlašovací stránku, kde se lze přihlásit nebo zaregistrovat nový účet. Pro úspěšnou registraci je potřeba zadat platnou e-mailovou adresu, uživatelské jméno a heslo. Uživatel má také možnost vybrat si, zda si má prohlížeč zapamatovat heslo pro příští návštěvy aplikace. Po úspěšné registraci je uživatel přesměrován do samotného uživatelského rozhraní aplikace.

Pokud uživatel již účet má, může se přihlásit zadáním e-mailové adresy a hesla. V případě zadání neplatných údajů je uživatel informován o chybě a je vyzván k opětovnému zadání.

5.3.1 Anonymní odesílání přihlášek

V případě, že je v nastavení aplikace povolena možnost přijímání přihlášek bez vytváření účtu, lze úspěšně odeslat přihlášku i bez přihlášení. V tomto případě odesílatele přihlášky ale nelze identifikovat, není tedy možné přihlášku spojit s konkrétním uživatelem, což znamená, že odeslané přihlášky nelze později upravovat ani prohlížet. Tato funkce je určena pro případy, kdy je potřeba umožnit odesílání přihlášek i pro uživatele, kteří nechtějí nebo nemohou vytvořit účet. V případě povolení této funkce se pod tlačítkem na registraci objeví možnost odeslat přihlášku bez registrace, což otevře jednorázový formulář.

5.3.2 Autentizace a autorizace pomocí knihovny Better Auth

Pro implementaci přihlašovacího formuláře, autentizace i autorizace byla zvolena knihovna Better Auth. Ta nabízí jednoduché a rychlé řešení přihlašování v aplikacích postavených nejen na Reactu a Next.js. Better Auth využívá session-based autentizaci, takže se do cookies ukládá session identifikátor, který server ověřuje při každém požadavku [24].

Knihovna je modulární a podporuje přidávání *pluginů*, které rozšiřují její funkce [24]. Jedním z těchto pluginů je i plugin pro podporu RBAC, který umožňuje definovat různé role uživatelů a jejich oprávnění v aplikaci. Struktura rolí v aplikaci byla rozdělena do tří úrovní, které odpovídají různým typům uživatelů a jejich oprávněním:

- `guest` (žadatel) – role pro běžného uživatele, tento uživatel vidí pouze do žadatelského rozhraní a nemá přístup k většině serverové API.
- `user` (vychovatel) – role pro vychovatele domova mládeže, uživatel s touto rolí vidí administrátorské rozhraní a má částečný přístup k serverovému API. Některé funkce, jako například konfigurace aplikace, archivace, nebo správa uživatelů jsou omezeny.
- `admin` (administrátor – hlavní vychovatel) – role pro hlavního vychovatele domova mládeže, uživatel s touto rolí má plný přístup k administrátorskému rozhraní a serverovému API. Může spravovat uživatele, nastavovat konfiguraci aplikace a provádět další administrativní úkony.

V případě běhu aplikace s nastavením přijímání přihlášek bez nutnosti registrace (anonymní režim), jsou práva anonymního uživatele omezena na minimum – pouhé odesílání přihlášek.

5.3.3 Middleware pro ochranu veřejné API

Je nutno zmínit, že aplikace obsahuje API, k němuž lze přistupovat z veřejné sítě. Toto API je vygenerované Next.js ze serverových akcí a je tedy přístupné z klientské části aplikace. API je nutno ochránit před neoprávněným přístupem, což je zajištěno pomocí knihovny Next-Safe-Action. Jedná se o knihovnu, která abstrahuje serverové akce v Next.js a přidává jim možnost validace, zachycování chyb a zachycování požadavků za chodu [25].

V konfiguraci Next-Safe-Action lze použít funkci `.use()` pro definování *middleware* (prostředník pro zachycování požadavků). Tento *middleware* je vykonán před samotnou serverovou akcí a může být použit pro různé účely, jako je kontrola autentizace uživatele, logování požadavků, či manipulace s daty požadavku.

5.4 Rozhraní pro žadatele

V režimu žadatele, tedy uživatele s rolí `guest`, má uživatel přístup k velmi omezeným funkcím samotné aplikace.

5.4.1 Boční navigační menu

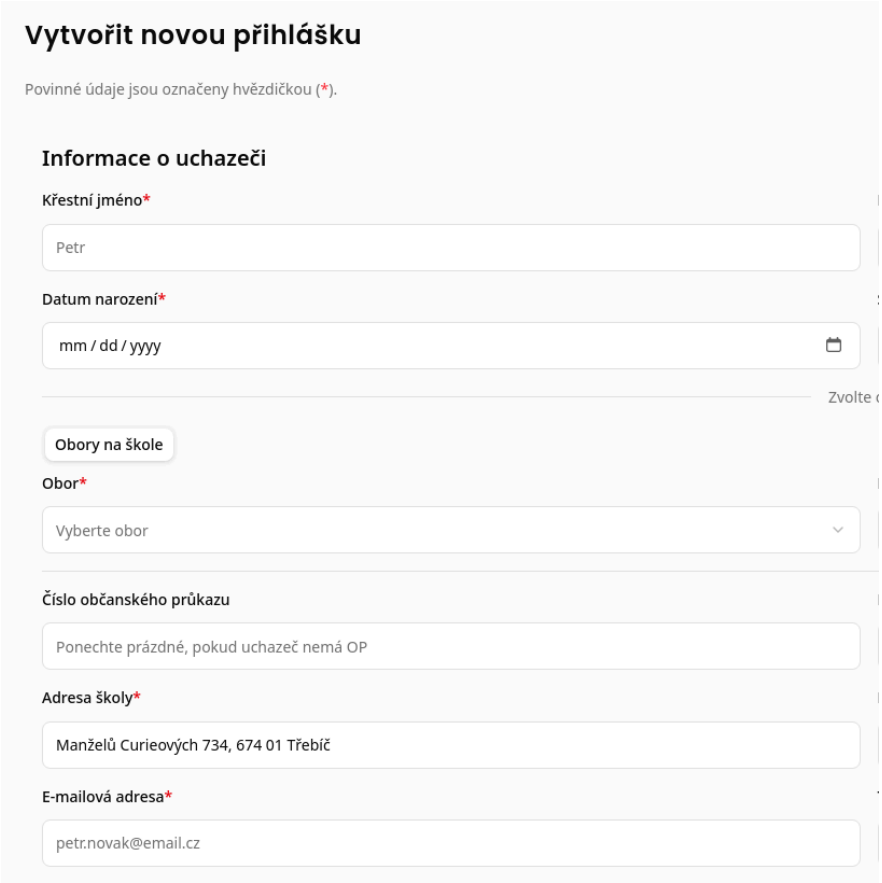
Boční navigační menu je hlavním navigačním prvkem aplikace. Umožňuje uživateli přístup k různým sekcím aplikace, jako je přehled přihlášek, možnosti uživatele, nebo odhlášení z aplikace. Navigační menu je navrženo tak, aby bylo responzivní pro různé velikosti obrazovky a zařízení.

5.4.2 Sekce „Moje přihlášky“

V sekci „Moje přihlášky“ lze vytvářet nové přihlášky, prohlížet si již vytvořené přihlášky a sledovat jejich stav. Vytvořené přihlášky jsou rozděleny do zobrazovacích karet, které obsahují rozkliknutelné karty s přehledem informací o přihlášce, jako je datum vytvoření, stav přihlášky a počet získaných bodů, a další.

5.4.3 Formulář pro vytvoření přihlášky

Formulář pro vytvoření přihlášky je dostupný po kliknutí na tlačítko *Nová přihláška* v sekci „Moje přihlášky“. K formuláři lze přistoupit pouze v případě, že nejsou splněny žádné podmínky pro zamezení přístupu k formuláři (např. uzávěrka přihlášek, či globální zamezení přístupu pro uživatele s rolí `guest`).



Obrázek 8: Část formuláře pro vytvoření přihlášky s několika sekcemi a různými typy vstupních polí

Formulář obsahuje několik sekcí, které pokrývají různé části přihlášky: údaje o žadateli, údaje o zákonných zástupcích a další otázky týkající se přihlášky. Každá sekce obsahuje různé typy vstupních polí, jako jsou textová pole, výběrové seznamy, přepínače a další. Pro postoupení do další sekce je vždy potřeba vyplnit všechna povinná pole tak, aby podléhala schématu validace. V případě špatně vyplněného pole je uživatel upozorněn a pole musí opravit. Po úspěšném vyplnění všech sekcí a odeslání formuláře je přihláška uložena do databáze a uživatel je přesměrován zpět do sekce „Moje přihlášky“, kde může sledovat stav své přihlášky. Přihlášku po odeslání již není možné upravovat.

Při každém odeslání formuláře je formulář zařazen do ročníku, jenž je aktuálně otevřen pro přijímání přihlášek. K rodnému číslu žadatele je také přiřazeno *evidenční číslo*, které je automaticky vygenerováno (nebo při opětovném podání přihlášky znovu použito) na základě počtu již přijatých přihlášek v daném ročníku (viz Kapitola 5.5.7).

5.4.4 Nastavení profilu

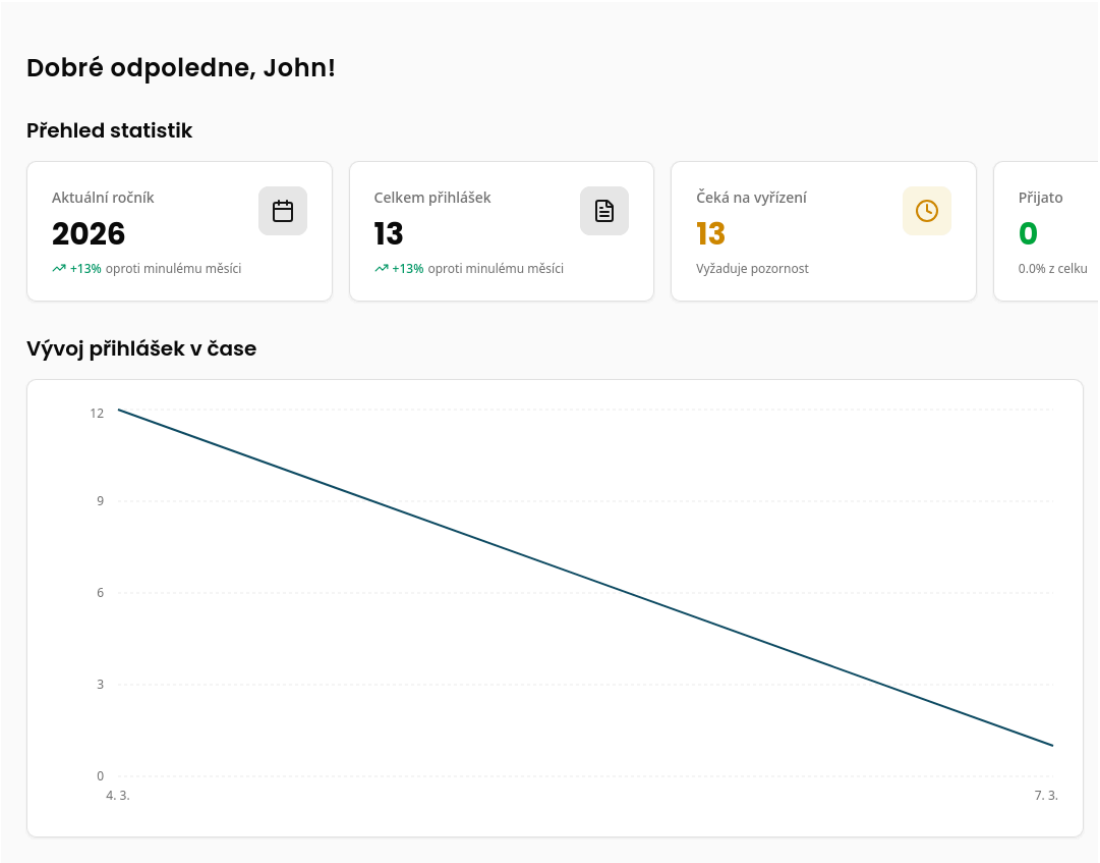
Sekce je určena pro správu uživatelského profilu. Uživatel zde může měnit své osobní údaje, jako je jméno, e-mailová adresa a heslo. Pro změnu hesla je potřeba zadat aktuální heslo a nové heslo.

5.5 Rozhraní pro vychovatele

Rozhraním pro vychovatele se rozumí administrativní část aplikace, která umožňuje spravovat přihlášky a vykonávat další administrativní úkony. Toto rozhraní je pouhým rozšířením uživatelského rozhraní pro žadatele, tím pádem má vychovatel stále přístup k podávání přihlášek a ostatním úkonům, které by za normálních okolností žadatel měl.

5.5.1 Úvodní přehled

Po přihlášení do administrativního rozhraní se vychovatelům zobrazí přehledná úvodní stránka, která obsahuje souhrn aktuálního ročníku. Mezi informacemi, které jsou zde zobrazeny, patří zejména statistiky o počtu přihlášek v aktuálním školním roce, graf zobrazující počet přihlášek v jednotlivých dnech, či koláčový graf zobrazující rozdělení přihlášek podle stavů přihlášek.



Obrázek 9: Úvodní přehled pro vychovatele s grafy a statistikami o aktuálním ročníku

5.5.2 Nastavení chování aplikace

Nastavení aplikace se nadále dělí na několik podčástí (sekcí). Většina těchto nastavení je dostupná pouze pro uživatele s rolí `admin` (hlavní vychovatel).

Sekce „Obecné“

Sekce obsahuje hlavní nastavení samotné aplikace, mezi některé vybrané konfigurovatelné možnosti patří:

- **Přístup k přihlašovacímu formuláři** – kontroluje nastavení přístupu k přihlašovacímu formuláři a je omezen na následující hodnoty:
  - „Otevřeno pro všechny (ignorovat uzávěrku)“ – umožňuje přístup k formuláři všem přihlášeným uživatelům.
  - „Otevřeno pro vychovatele (ignorovat uzávěrku)“ – přístup k přihlášce je možný pouze uživatelům s rolí *user* a vyšší.

  - „Uzavřeno po datu konce přihlašování“ – obsah je dostupný pouze do vypršení předem stanovené lhůty.
  - „Uzavřeno“ – k obsahu nemá přístup žádný typ uživatele.
- **Datum konce přihlašování** – nastavením data a času znemožníte přístup k formuláři po daném termínu. Toto pravidlo však vstoupí v platnost pouze v případě, že je přístup k formuláři nastaven na „Uzavřeno po datu konce přihlašování“.
- **Název domény** – toto pole je pouze estetické. Vzhledem k univerzálnímu návrhu aplikace bylo přidáno toto pole, aby žadatelé dokázali jednoduše odlišit, pro jaké zařízení aplikaci využívají.
- **Povolit vlastní obory** – tato možnost umožní žadatelům zadat vlastní studijní obor v případě, že nestudují na žádném z předdefinovaných oborů. Pokud je tato možnost zakázána, musí žadatelé vybrat obor ze seznamu předdefinovaných oborů.
- **Potvrzení při odeslání přihlášky** – slouží jako text, který musí uživatelé odsouhlasit před odesláním samotné přihlášky.

Sekce „Školní roky“

V této sekci lze spravovat školní roky. Školní rok je základní jednotkou pro organizaci přihlášek. Každý školní rok je identifikován svým počátečním rokem. Roky jsou však zobrazovány ve formátu „počáteční rok / následující rok“ (např. „2023/2024“).

Každý školní rok má několik vlastností, které lze nastavit:
- Stav archivace – pokud je školní rok archivován, již není možné přidávat nové přihlášky do tohoto roku, ale stále je možné prohlížet stávající přihlášky a exportovat je.
- Je-li aktuální – tento stav označuje, že se jedná o aktuální školní rok, do kterého se přidávají nové přihlášky. V aplikaci může být pouze jeden aktuální školní rok.

Sekce „Studijní obory“

Sekce sloužící pro správu studijních oborů. Obsahuje tabulku s jednotlivými obory. Každý obor má jméno, krátkou formu jména, délku studia a možnost nastavení, zda-li se jedná o obor, ve kterém začíná výuka brzy (tento fakt poté ovlivňuje samotné bodování přihlášky).

Sekce „Účty“

Sekce pro správu uživatelských účtů. Uživatelé mohou být přidáváni, odebíráni a lze upravovat jejich role.

Sekce „Generace PDF“

Sekce určená pro správu šablony PDF, která se používá pro generaci PDF dokumentů z přihlášek. Šablona je nahrána do systému a obsahuje formulářová pole pro všechny potřebné údaje z přihlášky. Tato pole jsou předem definována webovou aplikací a jsou následně vyplňována daty z přihlášky při generování PDF (více v Kapitola 5.5.5).

- `ID přihlášky`: Unikátní identifikátor přihlášky
- `Stav`: Aktuální stav přihlášky
- `Soukromá poznámka`: Interní poznámka viditelná pouze pro administrátory
- `Body`: Počet bodů přidělených přihlášce
- `Důvod zamítnutí`: Důvod zamítnutí přihlášky
- `Zdravotní poznámka`: Poznámka týkající se zdravotního stavu uchazeče
- `Speciální požadavky`: Specifické požadavky uchazeče
- `Koníčky`: Koníčky a zájmy uchazeče
- `Veřejná poznámka`: Poznámka viditelná pro uchazeče
- `Vyloučen dříve`: Zda byl uchazeč v minulosti vyloučen nebo bylo zahájeno správní řízení

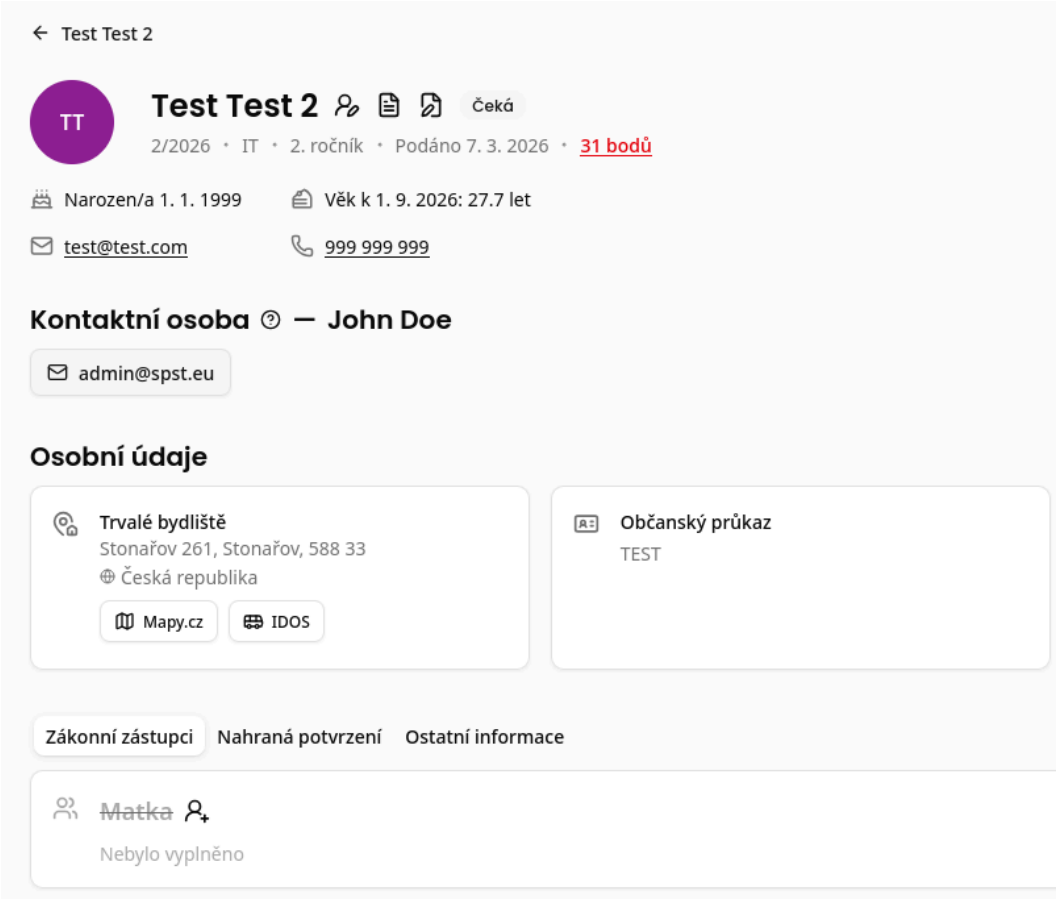
Obrázek 10: Vybraná jména polí, která lze do šablony PDF vložit pro následné vyplnění daty z přihlášky.

5.5.3 Zobrazení přijatých přihlášek

Vychovatelé mají přístup k přehlednému seznamu všech přihlášek. Seznam je zde implementován pomocí přehledné tabulky s podporou stránkování, filtrování a vyhledávání. Každá přihláška je zobrazena v řádku tabulky s možností rozkliknutí pro zobrazení detailních informací o přihlášce. Z této stránky lze také přihlášky mazat či provádět hromadné akce, jako je export do PDF pro více přihlášek najednou, nebo změna stavu více přihlášek najednou.

5.5.4 Detaily přihlášky

Každou přihlášku lze rozkliknout pro zobrazení detailních informací. V detailu přihlášky jsou zobrazeny všechny informace, které žadatel zadal při vyplňování přihlášky, včetně automaticky vygenerovaného bodového ohodnocení a stavu přihlášky. Vychovatel zde má také možnost měnit stav přihlášky (např. přijatá, zamítnutá) a přidávat poznámky. V případě, že uživatel, který přihlášku vytvořil, špatně vyplnil libovolný údaj, má možnost údaj v přihlášce opravit.



Obrázek 11: Stránka s detailem přihlášky, kde jsou zobrazeny všechny informace o přihlášce a možnosti pro změnu stavu přihlášky a přidávání poznámek.

Mezi vybrané funkce, které jsou dostupné v detailu přihlášky, patří například možnost zobrazení trvalého bydliště v aplikaci Mapy.com[6] nebo v aplikaci IDOS[7] (pro kontrolu vzdálenosti mezi trvalým bydlištěm a internátem). Kliknutím na individuální e-mailové

[6] https://mapy.com/
[7] https://idos.cz/

adresy lze také odesílat e-maily přímo z aplikace, což usnadňuje komunikaci s žadateli a zákonnými zástupci.

5.5.5 Export přihlášek do PDF

Jednou z klíčových funkcí pro vychovatele je možnost exportovat přihlášky do PDF formátu. Tento export umožňuje snadné sdílení a archivaci přihlášek mimo samotnou aplikaci. Aktuální způsob řešení využívá funkce standardu PDF, polemi AcroForms, které umožňují definovat interaktivní formulářová pole přímo v PDF dokumentu. Tato pole jsou následně vyplňována daty z přihlášky při generování PDF. Tato pole jsou podporávna většinou moderních PDF editorů [26].

Export přihlášky do PDF formátu je ale poněkud záludná záležitost, jelikož generování PDF na straně serveru v prostředí Node.js není příliš běžné ani efektivní. Aktuálně existují dvě možná řešení pro generování PDF z předhotovené šablony, které by vyhovovali aktuálnímu systému na domově mládeže: přes vyplňování buněk v XLSX (což je původní formát, ve kterém je přihláška tvořena) a přes vyplňování formulářových polí přímo v PDF. V jednoduchosti, efektivitě zpracování, kvalitě a univerzalitě výsledného PDF se ukázalo druhé řešení jako lepší, a proto také bylo zvoleno pro implementaci této funkce. Pro tento účel byla zvolena knihovna PDF--LIB, která umožňuje jednoduchou manipulaci s PDF dokumenty, včetně vyplňování formulářových polí [27]. Celý proces generování PDF probíhá následovně:

- Předem je vytvořena šablona PDF, která obsahuje formulářová pole pro všechny potřebné údaje z přihlášky. Tato pole jsou předem definována webovou aplikací. Šablona je nahrána do systému přes nastavení aplikace.
- Pro vygenerování PDF konkrétní přihlášky, či sady přihlášek lze použít tlačítko pro export do PDF, které se nachází jak ve stránce pro detaily přihlášky, tak i v přehledu přihlášek. Po kliknutí na tlačítko se spustí proces generování PDF, který načte šablonu PDF, vyplní formulářová pole daty z přihlášky a vygeneruje výsledný PDF dokument, který je následně odeslán uživateli ke stažení.

Interně je pro generování PDF vytvořena speciální API, která přijímá ID přihlášky (nebo skupiny ID) a vrací vyplněný PDF dokument. Všechny PDF jsou zpracovány v samostatném vlákně (*worker thread*), aby nedocházelo k blokování hlavního vlákna serveru a tím i zhoršení výkonu aplikace. Všechny přihlášky se zároveň ukládají do

lokální cache, aby se zrychlil přístup k již vygenerovaným PDF dokumentům (cache se automaticky přepisuje při případných změnách přihlášek, aby se zajistila aktuálnost dat).

Hlavní výhodou užití knihovny PDF-LIB oproti řešení s XLSX je fakt, že XLSX řešení má podstatně horší časovou komplexitu ($O(\text{Řádky} \cdot \text{Sloupce})$ – nutnost procházet všechny buňky), zatímco PDF-LIB nabízí konstantní časovou komplexitu ($O(1)$ – přímý přístup pomocí mapy [26]) pro vygenerování jednoho PDF dokumentu, což je zásadní pro zajištění rychlé odezvy aplikace. Při testování obou řešení se ukázalo, že průměrná odezva pro vygenerování PDF pomocí PDF-LIB byla přibližně 600 ms, zatímco pro řešení s XLSX to bylo přibližně 5.5 sekundy.

5.5.6 Archivace

V systému lze školní roky archivovat, což znamená, že přihlášky v daném školním roce již nebudou moci být upravovány ani přidávány nové přihlášky. Archivaci školních roků lze spravovat v sekci nastavení aplikace. Nelze aktivovat ročník, který je nastaven jako výchozí.

Archivace školního roku nemá vliv na již vygenerovaná evidenční čísla přihlášek, ta zůstávají nadále platná a jsou spojena s daným ročníkem. Archivace slouží tedy převážně k ochraně dat před nechtěnými úpravami a organizaci dat v systému.

5.5.7 Evidenční čísla

Evidenční čísla jsou unikátní identifikátory uchazečů, které mohou být generovány při odeslání přihlášky. Evidenční číslo je tvořeno kombinací prvního ročníku, ve kterém byla pro dané rodné číslo vygenerována přihláška, a pořadového čísla přihlášky v daném ročníku. To implikuje, že pro každé rodné číslo bude evidenční číslo vygenerováno pouze jednou a při opětovném podání přihlášky v dalším ročníku bude použito již existující evidenční číslo.

$$2025/1$$

Obrázek 12: Příklad evidenčního čísla přihlášky – první přihláška v roce 2025

V programu lze importovat existující evidenční čísla pro předem definovaná rodná čísla. Tento import je užitečný v případech, kdy docházelo k tvoření evidenčních čísel mimo systém (např. ručně) a je potřeba tato čísla synchronizovat s databází aplikace.

Import probíhá pomocí CSV souboru, který obsahuje 3 sloupce: rodné číslo, evidenční číslo a první ročník, ve kterém byla přihláška s daným rodným číslem odeslána (tento ročník musí v aplikaci předem existovat, jinak bude proces neúspěšný).

5.5.8 Massmail

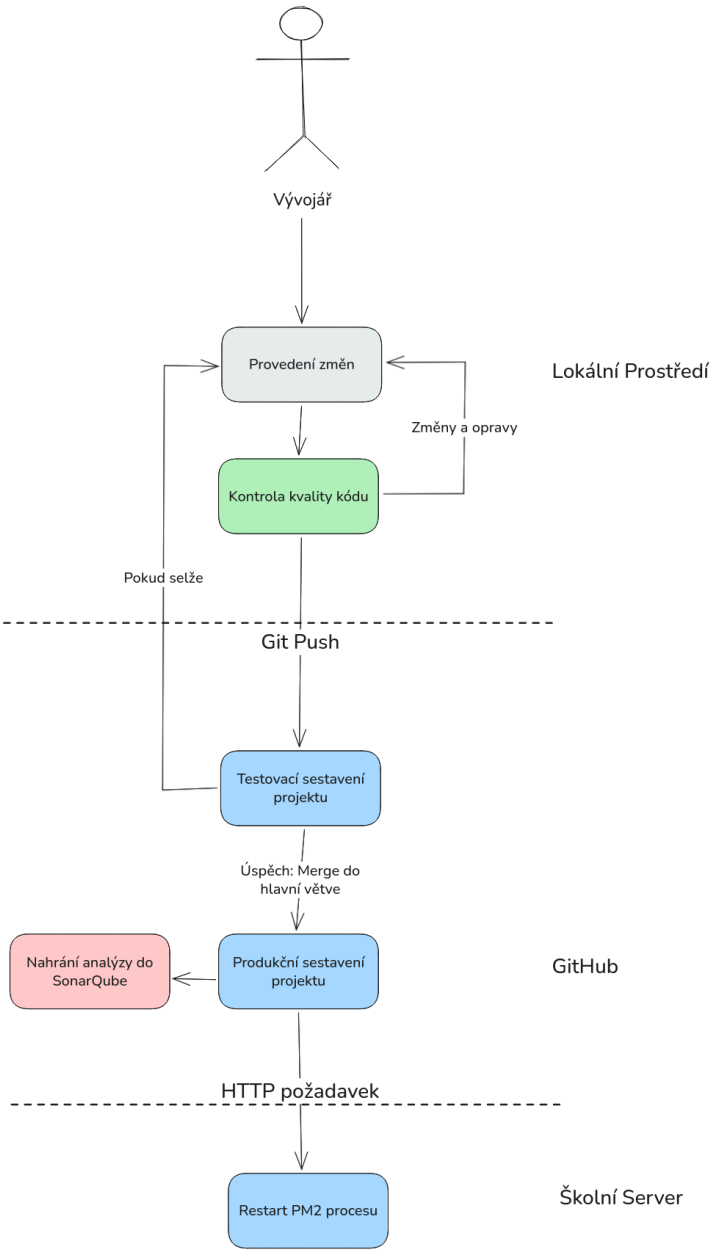
Massmail, neboli hromadná korespondence, je funkce, která umožňuje hromadné odesílání e-mailů všem žadatelům, nebo vybraným skupinám žadatelů na základě různých kritérií (např. stav přihlášky, ročník, atd.). Tato funkce je užitečná pro komunikaci s velkým počtem žadatelů najednou, například pro informování o změnách v přijímacím řízení, nebo pro zasílání potvrzení o přijetí přihlášky. K funkci lze přistoupit pomocí bočního navigačního menu. E-maily nabízí základní funkce formátování, jako je tučný text, kurzíva a odrážky.

5.5.9 Bodování přihlášek

Další důležitou funkcí v aplikaci je automatické bodování příchozím přihlášek. Přihlášky jsou po odeslání automaticky obodovány a jejich bodový stav se automaticky reflektuje v detailech, kde lze i body manuálně později přepočítat, samotní uchazeči s touto hodnotou však seznámeni nejsou, neboť se v jejich případě považuje za nerelevantní. Bodování je v implementováno přímo v aplikaci a pro jakoukoliv změnu je potřeba změnit aplikaci znovu sestavit. Tento stav není finální a v průběhu přechodu k větší univerzalitě je nutné aby tato funkce šla přímo konfigurovat z nastavení aplikace.

6 Vývoj a nasazení

Při tvorbě projektu by měl být kladen důraz na efektivní vývojový proces, který zahrnuje nástroje pro jednodušší lokální vývoj, zajištění kvality kódu, monitorování aplikace po nasazení a automatizaci nasazení. V následující kapitole je proto popsán přesný vývojový proces, nástroje použité pro zajištění kvality kódu a samotný automatizovaný proces nasazení aplikace na produkční server.



Obrázek 13: Schéma procesu vývoje aplikace

6.1 Vývojové nástroje

6.1.1 Docker

Ve vývojovém prostředí lze aplikaci i služby nutné pro její běh (např. databáze) spustit pomocí Dockeru. Pro tento účel je v kořenovém adresáři projektu umístěn soubor `docker-compose.dev.yml`, který definuje potřebné služby a jejich konfiguraci. Tento soubor lze použít pro rychlé a jednotné spuštění databáze Postgres a podpůrného SMTP serveru MailHog.

Společně se souborem `docker-compose.dev.yml` je v kořenovém adresáři projektu umístěn i soubor `Dockerfile`, který definuje sestavení samotné aplikace do Docker obrazu. Tento obraz je následně použit v `docker-compose.yml` pro spuštění aplikace v produkčním prostředí. Nutno podotknout, že na produkčním serveru je aplikace spuštěna přes *LXC kontejner*, nikoliv jako Docker kontejner. Toto rozhodnutí bylo učiněno na základě konzultace s administrátorem lokálního školního serveru.

6.1.2 Užití statických analýzátorů kódu

Pro zajištění kvality kódu, dodržování standardů programování a odhalování potenciálních chyb byly do vývojového procesu integrovány *nástroje pro statickou analýzu kódu*. Tyto nástroje analyzují zdrojový kód bez jeho spuštění a poskytují zpětnou vazbu vývojářům o možných problémech, jako jsou chyby v syntaxi, nedodržování konvencí, či potenciální bezpečnostní rizika, tímto se liší od analyzérů dynamických, jako jsou testy, nástroje pro analýzu výkonu (profilery) nebo ladicí nástroje (debuggery).

Biome

Biome je moderní nástroj pro statickou analýzu kódu, který podporuje různé programovací jazyky, včetně TypeScriptu. Biome nabízí funkce, jako je formátování kódu podle předem stanovených pravidel, detekce chyb v syntaxi, analýza kvality kódu podle definovaných standardů pro statickou analýzu webových aplikací[8] a další [28].

[8] Tato pravidla jsou často převzata z jiných nástrojů, jako je ESLint. Kompletní seznam pravidel a jejich odůvodnění lze nalézt v oficiální dokumentaci Biome.

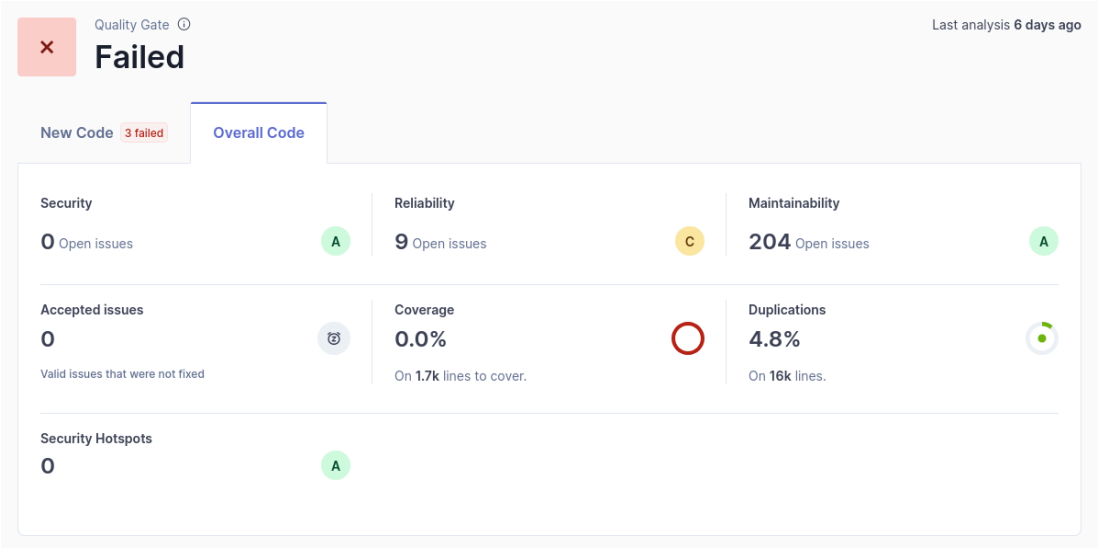
Součástí lokální konfigurace Biome je i integrace s verzovacím systémem Git pomocí *pre-commit hooku*. Z názvu je patrné, že se jedná o skript nebo jinou akci, která bude vykonána před tím, než dojde k vytvoření nového *commitu* v lokálním repozitáři. V případě Biome je tento *hook* použit pro automatické spuštění analýzy kódu a případné opravy nalezených problémů.

Další funkcí Biome je integrace s editory a vývojovými prostředími (jako je Visual Studio Code, nebo JetBrains WebStorm) pomocí rozšíření. Tato integrace umožňuje vývojářům získávat zpětnou vazbu o kvalitě kódu přímo během psaní kódu, což usnadňuje dodržování standardů a zlepšuje kvalitu kódu již v raných fázích vývoje, ještě než je samotný nástroj spuštěn manuálně.

Celé nastavení Biome je uloženo v kořenovém adresáři projektu v souboru `biome.json`, kde jsou definována pravidla pro analýzu kódu, formátování a další.

SonarQube

SonarQube je platforma pro inspekci kvality kódu v softwarových projektech. Podporuje širokou škálu programovacích jazyků, včetně TypeScriptu, a nabízí funkce jako je analýza kódu, detekce chyb, sledování metrik kvality a další [29].



Obrázek 14: Souhrn analýzy zdrojového kódu v SonarQube

Pro integraci SonarQube do vývojového procesu byl SonarQube nasazen na vlastní server a byl vytvořen nový projekt. Pro integraci mezi repozitářem na platformě

GitHub a SonarQube byla vytvořena nová *GitHub Action*, tedy automaticky vykonaná činnost. Ta je nastavena tak, aby se spustila při každé aktualizaci zdrojového kódu na hlavní větvi repozitáře. Tato akce provede analýzu kódu pomocí SonarQube a výsledky jsou následně odeslány na server SonarQube, kde jsou dostupné pro další přehledy a analýzy.

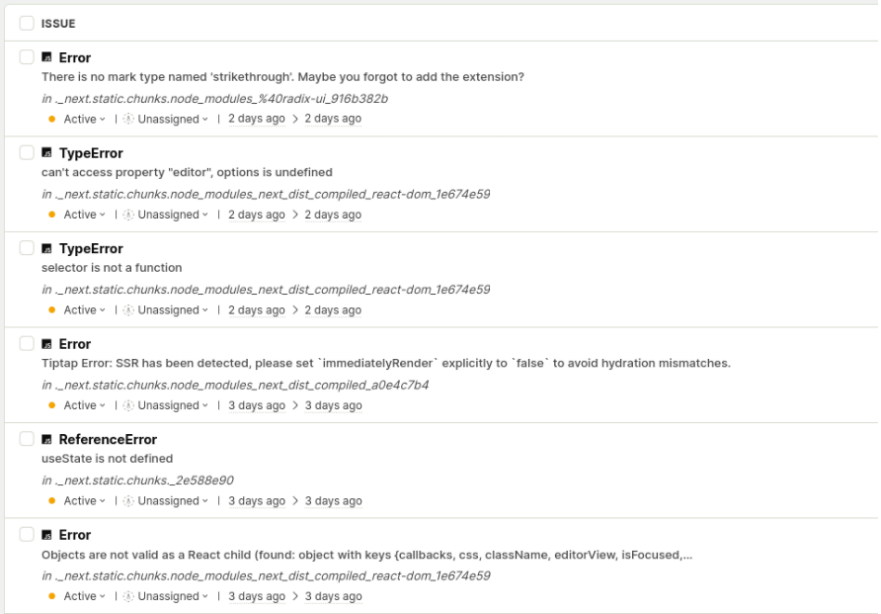
6.1.3 Aktivní monitorování

Mezi nástroje pro aktivní monitorování webových aplikací řadíme nástroje, které aktivně sledují chování aplikace v reálném čase a poskytují zpětnou vazbu o výkonu, chybách a dalších důležitých aspektech aplikace. Tyto nástroje pomáhají vývojářům identifikovat a řešit problémy (například sledováním kroků uživatele, které vedly k vyvolání chyby), optimalizovat výkon a zlepšovat uživatelskou zkušenost za běhu aplikace.

PostHog

PostHog je open-source platforma pro analýzu chování uživatelů a aktivní monitorování webových aplikací. Umožňuje vývojářům sledovat interakce uživatelů, analyzovat chování aplikace, zajišťovat neodlaďené chyby a získávat cenné informace pro optimalizaci výkonu a zlepšení uživatelské zkušenosti [30].

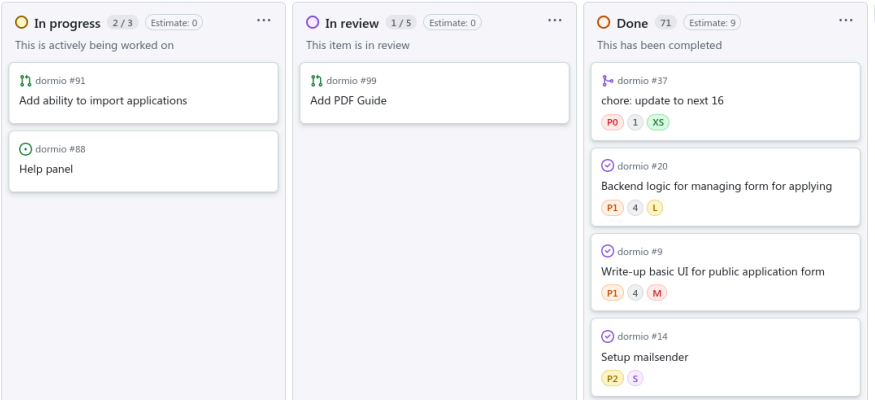
PostHog byl do vývojového procesu integrován použitím oficiální knihovny. Hlavním cílem integrace bylo vytvořit možnost získávání zpětné vazby o chování uživatelů a sledování chyb, které by mohly nastat během používání aplikace. Tato integrace umožňuje lépe porozumět chybám a problémům, které uživatelé mohou zažívat. Integrace zároveň pomáhá rychleji problémy opravovat a případně i předcházet jejich vzniku v budoucnu.



Obrázek 15: Panel pro sledování nastalých problémů při běhu projektu v platformě PostHog

6.2 Plánování vývoje pomocí GitHub Projects

Správné plánování vývoje je jednou z klíčových činností pro úspěšný a nezanedbaný vývoj jakéhokoliv softwarového projektu. Pro tento účel byla zvolena platforma GitHub Projects, která umožňuje vytváření projektů, úkolů a sledování jejich stavu přímo v rámci repozitáře na GitHubu [31].



Obrázek 16: Plánovací tabulka pro vývoj aplikace na platformě GitHub Projects

Struktura plánovací tabulky byla rozdělena do několika sloupců:

- *Backlog* – sloupec pro úkoly, které jsou naplánovány, ale ještě nebyly zahájeny a aktuálně nejsou žádné kapacity pro jejich řešení.
- *Ready* – sloupec pro úkoly, které jsou připraveny k řešení a čekají na zahájení práce.

- *In Progress* – sloupec pro úkoly, na kterých se aktuálně pracuje.
- *In Review* – sloupec pro úkoly, které byly dokončeny a čekají na schválení (např. kontrolou kódu).
- *Done* – sloupec pro úkoly, které byly dokončeny a schváleny.

Každý z úkolů je propojen s příslušnou *issue* (problémem) nebo *pull requestem* (žádostí o sloučení kódu) v repozitáři, což umožňuje snadné sledování pokroku a stavu jednotlivých úkolů přímo z plánovací tabulky. Zároveň je ke každému úkolu možné přiřadit různé typy štítků (labels), které pomáhají kategorizovat úkoly podle jejich povahy (např. bug, feature, enhancement) a priority.

Samostatné *issues* a *pull requesty* jsou pak spravovány pomocí standardních funkcí platformy GitHub a lze je opět kategorizovat do štítků, ty ale nyní slouží pro identifikaci typu problému či změny, kterou daný *issue* nebo *pull request* řeší (například problém se štítkem `ui/ux` značí problém, který se týká uživatelského rozhraní).

6.3 Verzovací systém Git

Git je distribuovaný verzovací systém, který umožňuje sledování změn v souborech a koordinaci práce mezi více vývojáři na jednom projektu [32]. Pro tento projekt byl zvolen Git jako hlavní nástroj pro správu verzí kódu, jelikož nabízí širokou škálu funkcí pro efektivní spolupráci a správu kódu, zároveň je dobře integrován s platformou GitHub, která slouží jako hostitelská služba pro repozitář projektu. Git zároveň patří mezi nejrozšířenější verzovací systémy, které jsou v současnosti používány ve vývoji softwaru [9].

6.4 GitHub Actions

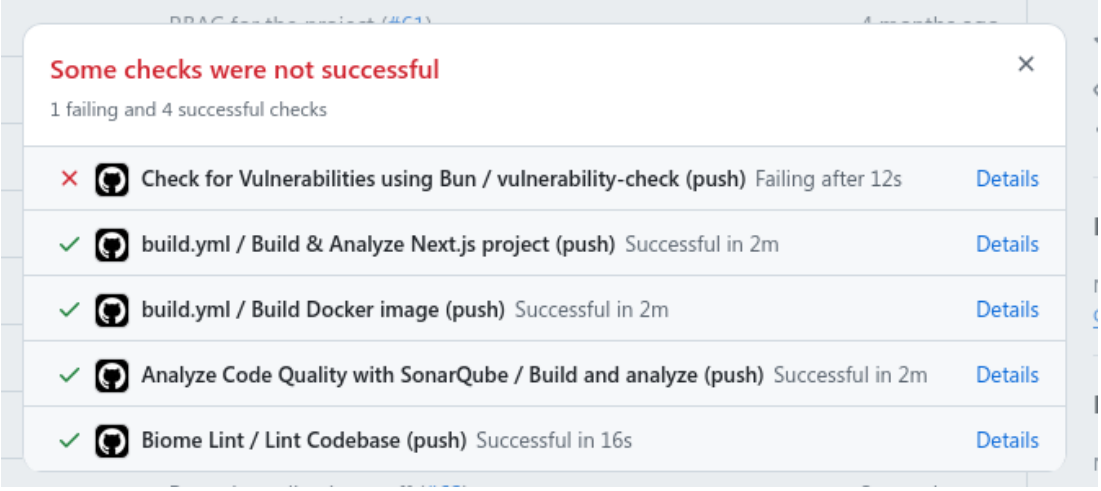
GitHub Actions je sada nástrojů pro automatizaci pracovních postupů přímo v rámci platformy GitHub. Jedná se o takzvané *CI/CD* (Continuous Integration/Continuous Deployment) nástroje, které umožňují automatizovat různé úkoly, jako je testování kódu, nasazení aplikace, nebo jiné opakující se úkoly [33].

V tomto projektu by se použité GitHub akce daly rozdělit do 2 kategorií:

1. Akce pro zajištění kvality kódu – tyto akce jsou nastaveny tak, aby se spouštěly při každé aktualizaci kódu, jedná se o kontrolu sestavení projektu, spuštění testů

a analýzu kódu pomocí SonarQube. Zároveň akce kontroluje zranitelnosti v závislostech projektu.

2. Akce pro nasazení aplikace – tyto akce jsou nastaveny tak, aby se spouštěly při každé aktualizaci hlavní větve repozitáře a zajišťují automatizované nasazení aplikace na produkční server.



Obrázek 17: Ukázka výpisu spuštěných akcí v GitHub Actions

6.5 Nasazení na produkční server

Nasazení aplikace na produkční systém probíhá pomocí automatizovaného procesu, který je spuštěn při každé aktualizaci hlavní větve v repozitáři na platformě GitHub. Tento proces je implementován pomocí GitHub Actions.

V prvním kroku procesu nasazení je spuštěna akce, která ověří spustitelnost kódu testovacím sestavením. Pokud je sestavení úspěšné, akce odešle HTTP požadavek na otevřené API jednoduchého webového serveru běžícího na produkčním serveru. Tento požadavek slouží jako spouštěč skriptu, který je zodpovědný za znovunasazení aplikace. Skript následně vykoná následující kroky, přesně v tomto pořadí:

- Zastaví běžící instance aplikace (pokud existuje), vypne pm2 proces.
- Aktualizuje zdrojový kód aplikace uložený na serveru pomocí `git pull` pro získání nejnovějších změn z repozitáře.
- Nainstaluje všechny potřebné závislosti (pokud došlo k jejich změně).
- Aplikuje případné změny v databázovém schématu pomocí migrací.
- Nastartuje aplikaci pomocí pm2 a znovu ji zpřístupní uživatelům.

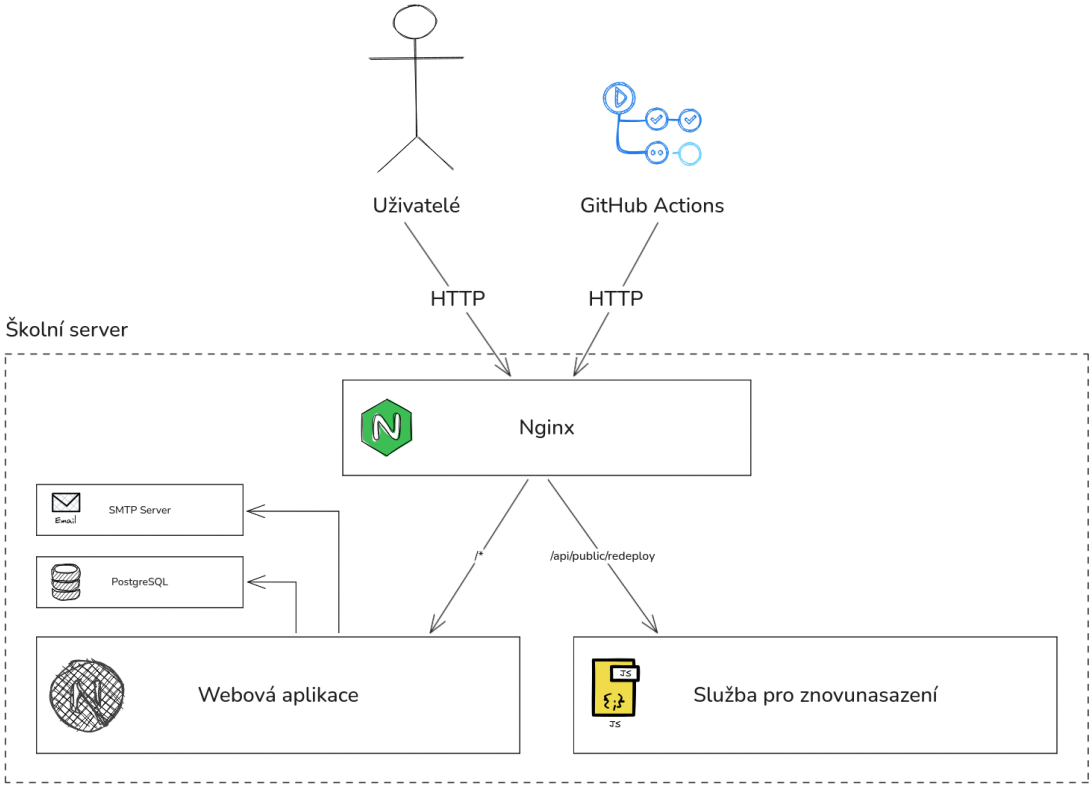
6.5.1 Inicializace aplikace

Při každém restartu aplikace dojde k *bootstrappingu* (inicializaci) aplikace. Tento proces zahrnuje kroky nutné pro funkci aplikace, jako je napojení na databázi, načtení a validace konfiguračních proměnných (ze souboru `.env`), či vytvoření vitálních struktur potřebných pro běh aplikace – například vytvoření výchozího administrátorského účtu, pokud v databázi ještě žádný neexistuje (údaje jsou převzaty z proměnných prostředí).

6.5.2 Architektura produkčního serveru

Produkční server je hostován jako LXC kontejner na hlavním školním serveru. Tento kontejner je nakonfigurován tak, aby poskytoval izolované prostředí pro běh aplikace, což zajišťuje bezpečnost a stabilitu aplikace. Kontejner obsahuje všechny potřebné závislosti a konfigurace pro běh aplikace, včetně databázového serveru PostgreSQL a přístupu na SMTP server pro odesílání e-mailů přes školní doménu.

Pro přístup k aplikaci z veřejné sítě je na školním serveru nastavena služba Nginx, která poskytuje reverzní proxy pro směrování HTTP požadavků na správný port, na kterém aplikace běží. V případě služby pro znovunasazení je pak v konfiguraci reverzní proxy nastaveno směrování požadavků na specifickou cestu a přístup je omezen na klíč pro ověření. Pro bezpečnostní účely je také implementována cesta pro *kill-switch*, která umožňuje pouhým posláním HTTP požadavku na specifickou cestu okamžitě vzdáleně zastavit běh aplikace, což je užitečné v případě zjištění závažné chyby, nebo bezpečnostního incidentu.



Obrázek 18: Schéma architektury produkčního serveru

6.6 Praktické testování

Důležitou částí vývoje aplikace bylo zajištění, že ji budou uživatelé (vychovatelé i žadatelé) skutečně používat a že bude dostatečně intuitivní. Společně s několika vychovateli byla proto vybrána skupinka studentů, kteří měli možnost systém otestovat. Cílem testování bylo hlavně získat zpětnou vazbu o uživatelské zkušenosti, identifikovat případné problémy a nedostatky v aplikaci a získat cenné informace, které by mohly být použity pro další vylepšení a zvýšení UX.

Testování probíhalo v několika iteracích, přičemž každá iterace zahrnovala jinou cílovou skupinu uživatelů (vychovatelé, rodiče, studenti…). Z každé iterace byla zpětná vazba využita pro vylepšení aplikace.

Závěr

Během vývoje této aplikace a psaní této práce jsem se setkal s mnoha výzvami, které mě přiměly k hlubšímu zamyšlení nad různými aspekty vývoje webových aplikací, jako je návrh uživatelského rozhraní, výběr vývojových nástrojů, zajištění kvality kódu a další. Tento projekt mi umožnil získat cenné zkušenosti a znalosti, které využiji i v budoucnu při vývoji dalších projektů.

Budoucí rozšíření

Aplikace v aktuálním stavu nabízí širokou škálu funkcí pro správu přihlášek na domovy mládeže. Nicméně stále existuje prostor pro další rozšíření a vylepšení, které by mohly funkcionalitu aplikace ještě více rozšířit a zlepšit uživatelskou zkušenost. Jedním z možných rozšíření je rozšíření systému na kompletní správu domovů mládeže, což by zahrnovalo nejen správu přihlášek, ale i další aspekty, jako je správa ubytování, vycházek, volnočasových aktivit a další. Dalším možným rozšířením je implementace pokročilých analytických nástrojů pro lepší porozumění datům o přihláškách a žadatelích, což by mohlo pomoci při optimalizaci přijímacího řízení a lepší vyhodnocování kritérií pro přijetí.

Všestrannost

Dalším aspektem aplikace byl i fakt, že jsem se ji snažil postavit co nejvíce univerzálně tak, aby ji bylo možné snadno přizpůsobit pro různé domovy mládeže. Toho bylo dosaženo například tím, že jsem do nastavení aplikace přidal možnost nastavit název domény, což umožňuje žadatelům snadno odlišit, pro jaké zařízení aplikaci využívají. Je třeba však zmínit, že některé funkce, jako například automatické bodování, univerzálně postaveny nejsou a případná univerzalita bude zajištěna později.

Seznam použité literatury

[1] GOOGLE LLC. *Google Forms: Online formuláře pro rychlé statistiky*. Online. Dostupné z: https://workspace.google.com/products/forms/. [citováno 2025-10-24]

[2] MALE, Vinay. Decoding Role-Based Access Control (RBAC). Online. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, roč. 11 (2025), s. 2082–2090. Dostupné z: https://doi.org/10.32628/CSEIT251112211. [citováno 2025-10-24]

[3] RICHARDS, M. a N. FORD. *Fundamentals of Software Architecture: An Engineering Approach*. Online. O'Reilly Media, 2020. ISBN 9781492043409. Dostupné z: https://books.google.cz/books?id=wa7MDwAAQBAJ. [citováno 2026-03-03]

[4] NIELSEN, Henrik; Jeffrey MOGUL; Larry M MASINTER; Roy T. FIELDING; Jim GETTYS et al. *Hypertext Transfer Protocol – HTTP/1.1*. Online. 1999-06. Dostupné z: https://doi.org/10.17487/RFC2616. [citováno 2026-03-18]

[5] OKTA, INC. *Introduction to JSON Web Tokens*. Online. 2026. Dostupné z: https://www.jwt.io/introduction#what-is-json-web-token-structure. [citováno 2026-02-05]

[6] GEEKSFORGEEKS. *Session vs Token Based Authentication*. Online. Dostupné z: https://www.geeksforgeeks.org/computer-networks/session-vs-token-based--authentication/. [citováno 2026-03-21]

[7] OWASP FOUNDATION. *Authentication Cheat Sheet*. Online. Dostupné z: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. [citováno 2026-03-21]

[8] MICROSOFT. *TypeScript: JavaScript with syntax for types*. Online. Dostupné z: https://www.typescriptlang.org/. [citováno 2025-10-24]

[9] STACK OVERFLOW. *Stack Overflow Developer Survey 2025*. Online. 2025. Dostupné z: https://survey.stackoverflow.co/2025/. [citováno 2026-01-28]

[10] MICROSOFT INC. *TypeScript Declaration Files: Do's and Don'ts*. Online. Dostupné z: https://www.typescriptlang.org/docs/handbook/declaration-files/do-s-and-don-ts.html. [citováno 2026-02-11]

[11] ECMA INTERNATIONAL. *ECMAScript®: The Standardized Scripting Language for Web Development*. Online. 2026. Dostupné z: https://www.ecma--international.org/publications-and-standards/standards/ecma-262/. [citováno 2026-02-11]

[12] MICROSOFT INC. *TypeScript Compiler Options: Target*. Online. Dostupné z: https://www.typescriptlang.org/tsconfig/#target. [citováno 2026-02-11]

[13] ECMA INTERNATIONAL. *ECMA-262: ECMAScript® 2015 Language Specification*. Online. 2015. Dostupné z: https://www.ecma-international.org/ecma-262/6.0/. [citováno 2026-02-11]

[14] VERCEL INC. *Next.js: The React Framework for the Web*. Online. Dostupné z: https://nextjs.org/. [citováno 2025-10-24]

[15] META PLATFORMS, INC. *React: The library for web and native user interfaces*. Online. Dostupné z: https://react.dev/. [citováno 2025-10-24]

[16] JOTAI CONTRIBUTORS. *Jotai: Primitive and flexible state management for React*. Online. Dostupné z: https://jotai.org/. [citováno 2026-03-03]

[17] META PLATFORMS, INC. *React Server Components*. Online. Dostupné z: https://react.dev/reference/rsc/server-components. [citováno 2025-10-30]

[18] VERCEL INC. *Server Actions in Next.js*. Online. Dostupné z: https://nextjs.org/docs/13/app/api-reference/functions/server-actions. [citováno 2025-10-24]

[19] PRISMA INC. *Prisma: Next-generation ORM for Node.js and TypeScript*. Online. Dostupné z: https://www.prisma.io/orm. [citováno 2025-10-30]

[20] THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL: Documentation*. Online. Dostupné z: https://www.postgresql.org/docs/. [citováno 2026-03-07]

[21] KANE, Andrew. *Open-source vector similarity search for Postgres*. Online. Dostupné z: https://github.com/pgvector/pgvector. [citováno 2026-03-07]

[22] TAILWIND LABS, INC. *Tailwind CSS: A utility-first CSS framework for rapid UI development*. Online. Dostupné z: https://tailwindcss.com/. [citováno 2025-11-12]

[23] COLINHACKS. *Zod: TypeScript-first schema validation with static type inference*. Online. 2023. Dostupné z: https://zod.dev/. [citováno 2026-02-11]

[24] BETTER AUTH INC. *Better Auth: The most comprehensive authentication framework for the web*. Online. Dostupné z: https://www.better-auth.com/. [citováno 2026-03-07]

[25] RANGHIERI, Edoardo. *Next Safe Action: Type-safe Server Actions for Next.js*. Online. Dostupné z: https://next-safe-action.dev/. [citováno 2025-12-25]

[26] PDF ASSOCIATION, INC. *ISO 32000-2:2020 (PDF 2.0)*. Online. 2020. Dostupné z: https://ia601607.us.archive.org/2/items/iso-32000-2-2020--sponsored/ISO_32000-2-2020_sponsored.pdf. [citováno 2026-03-18]

[27] DILLON, Andrew. *Create and modify PDF documents in any JavaScript environment*. Online. Dostupné z: https://pdf-lib.js.org/. [citováno 2026-03-18]

[28] BIOME DEVELOPERS AND CONTRIBUTORS. *Biome, toolchain of the web*. Online. Dostupné z: https://biomejs.dev/. [citováno 2026-01-27]

[29] SONARSOURCE SA. *SonarQube: Code quality and security*. Online. Dostupné z: https://www.sonarsource.com/products/sonarqube/. [citováno 2026-01-27]

[30] POSTHOG, INC. *PostHog: We make dev tools for product engineers*. Online. Dostupné z: https://posthog.com/. [citováno 2026-01-28]

[31] GITHUB, INC. *GitHub Projects: Project planning for developers*. Online. Dostupné z: https://github.com/features/project-management/. [citováno 2026-01-28]

[32] TORVALDS, Linus a Junio C. HAMANO. *Git: Fast Version Control System*. Online. Dostupné z: https://git-scm.com/. [citováno 2026-01-28]

[33] GITHUB, INC. *GitHub Actions: Automate, customize, and execute your software development workflows right in your repository*. Online. 2026. Dostupné z: https://github.com/features/actions. [citováno 2026-01-28]

Seznam obrázků

Obrázek 1 Vizualizace client-server architektury 14
Obrázek 2 Vizualizace komunikace mezi klientem a serverem pomocí HTTP protokolu 15
Obrázek 3 Vizualizace JWT autentizace 17
Obrázek 4 Vizualizace session-based autentizace 17
Obrázek 5 Vizualizace procesu transpilace TypeScriptu do JavaScriptu 20
Obrázek 6 Výstup příkazu `next build` zobrazující jednotlivé kroky kompilace a optimalizace kódu v Next.js 27
Obrázek 7 Databázový model aplikace jako Entity-Relationship Diagram 31
Obrázek 8 Část formuláře pro vytvoření přihlášky s několika sekcemi a různými typy vstupních polí 35
Obrázek 9 Úvodní přehled pro vychovatele s grafy a statistikami o aktuálním ročníku 37
Obrázek 10 Vybraná jména polí, která lze do šablony PDF vložit pro následné vyplnění daty z přihlášky. 39
Obrázek 11 Stránka s detailem přihlášky, kde jsou zobrazeny všechny informace o přihlášce a možnosti pro změnu stavu přihlášky a přidávání poznámek. 40
Obrázek 12 Příklad evidenčního čísla přihlášky – první přihláška v roce 2025 42
Obrázek 13 Schéma procesu vývoje aplikace 44
Obrázek 14 Souhrn analýzy zdrojového kódu v SonarQube 46
Obrázek 15 Panel pro sledování nastalých problémů při běhu projektu v platformě PostHog 48
Obrázek 16 Plánovací tabulka pro vývoj aplikace na platformě GitHub Projects . . 48
Obrázek 17 Ukázka výpisu spuštěných akcí v GitHub Actions 50
Obrázek 18 Schéma architektury produkčního serveru 52

Seznam tabulek

Tabulka 1 Zjednodušené porovnání řešení přihlašovacího procesu do domova mládeže 10
Tabulka 2 Porovnání standardů pro autentizaci a autorizaci 18
Tabulka 3 Porovnání vybraných aspektů skriptovacích jazyků a srovnání TypeScriptu a JavaScriptu 21

Seznam výpisů

Výpis 1 Ukázka práce s rozhraními – definice rozhraní a typu odkazujícího na vlastnost rozhraní . . . 20
Výpis 2 Ukázka komponenty v Reactu . . . 22
Výpis 3 Ukázka využití Context API pro správu stavu v Reactu . . . 23
Výpis 4 Ukázka využití knihovny Jotai pro správu stavu . . . 24
Výpis 5 Ukázka React Server Component a jejího použití s Suspense . . . 25
Výpis 6 Ukázka Server Action v Next.js . . . 26
Výpis 7 Ukázka komponenty v Next.js využívající Server Action . . . 26
Výpis 8 Ukázka schématu databáze v Prisma ORM s modelem `User` . . . 28
Výpis 9 Ukázka použití generovaného kódu pro práci s modelem `User` . . . 28
Výpis 10 Jednoduché stylování za použití frameworku TailwindCSS . . . 29
Výpis 11 Příklad užití knihovny Zod . . . 32

Seznam zkratek a symbolů

ACID **– Atomicity, Consistency, Isolation, Durability**: Sada vlastností zajišťujících spolehlivost transakcí v databázových systémech. 28

API **– Application Programming Interface**: Programové rozhraní, které definuje způsob komunikace mezi různými částmi aplikace nebo mezi různými službami.

CI/CD **– Continuous Integration / Continuous Deployment**: Soubor postupů a nástrojů pro průběžnou integraci změn, automatizované testování a nasazování aplikace.

CRUD **– Create, Read, Update, Delete**: Základní čtyři operace používané při práci s daty v databázových a aplikačních systémech.

JIT **– Just-In-Time**: Způsob kompilace nebo generování výstupu až v okamžiku potřeby, typicky pro optimalizaci výkonu a velikosti výsledku.

JSX **– JavaScript XML**: Syntaktické rozšíření JavaScriptu používané v Reactu pro zápis komponent podobný HTML.

JWT **– JSON Web Token**: Kompaktní tokenový formát používaný pro přenos informací o identitě a oprávněních uživatele.

LXC **– Linux Containers**: Technologie virtualizace na úrovni operačního systému pro izolovaný běh aplikací.

MVC **– Model-View-Controller**: Architektonický vzor oddělující datový model, prezentační vrstvu a řídicí logiku aplikace.

MVVM **– Model-View-ViewModel**: Architektonický vzor, který odděluje uživatelské rozhraní od aplikační logiky pomocí vrstvy ViewModel.

ORM **– Object-Relational Mapping**: Technika mapování objektů v programovacím jazyce na relační databázové tabulky.

RBAC **– Role-Based Access Control**: Model řízení přístupu, ve kterém jsou oprávnění přiřazena rolím a uživatelé získávají práva podle své role.

RLS **– Row-Level Security**: Mechanismus zabezpečení databáze umožňující omezit přístup k jednotlivým řádkům tabulky. 28

RSC **– React Server Components**: Typ React komponent vykonávaných na serveru, které umožňují načítat data bez přímého běhu v prohlížeči.

SMTP **– Simple Mail Transfer Protocol**: Síťový protokol používaný pro odesílání e--mailových zpráv.

***SSR* – Server-Side Rendering**: Technika vykreslování webových stránek na serveru před jejich odesláním klientovi.

***UUID* – Universally Unique Identifier**: Standardizovaný formát pro vytváření prakticky unikátních identifikátorů.

***UX* – User Experience**: Celkový uživatelský prožitek při používání aplikace, včetně srozumitelnosti a pohodlí ovládání.